

Министерство образования и науки Краснодарского края
ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ПРОФЕССИОНАЛЬНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ КРАСНОДАРСКОГО КРАЯ ЕЙСКИЙ
ПОЛИПРОФИЛЬНЫЙ КОЛЛЕДЖ
(ГБПОУ КК ЕПК)

Отделение педагогики

УЧЕБНАЯ ПРАКТИКА 3 КУРС V СЕМЕСТР

МЕТОДИЧЕСКИЕ РЕКОМЕНДАЦИИ

г. Ейск, 2025 г.

Содержание

Цель работы.....	5
Решаемые задачи.....	5
1 Теоретические сведения.....	5
1.1 Тестовая документация.....	5
1.2 Тест-свит (набор тестов).....	6
1.2.1 Примеры содержания наборов тест кейсов.....	6
1.2.2 Содержание тест-кейсов в тест-свитах.....	7
1.3 Тест-кейс (тестовый случай).....	9
1.3.1 Содержание тест-кейса.....	9
1.3.2 Примеры Тест-кейсов.....	10
1.3.3 Статусы прохождения тест-кейсов.....	11
1.3.4 Обозначение приоритетов.....	11
1.3.5 Дополнительные элементы.....	12
1.3.6 Шаблоны тест-кейсов для различных типов тестирования....	12
1.3.7 Высокоуровневый и низкоуровневый тест-кейсы.....	13
1.4 Чек-лист.....	14
1.4.1 Виды чек-листов.....	14
1.4.2 Пример чек-листа.....	15
1.5 Баг-репорт.....	15
1.5.1 Формат баг-репорта.....	15
1.5.2 Пример заполнения баг-репорта.....	16
2 Тестирование требований.....	18
2.1 Теоретические сведения.....	18
2.2 Критерии качества требований.....	18
2.3 Этапы тестирования требований.....	19
2.4 Техники тестирования требований.....	19
2.5 Примеры типичных дефектов.....	20

2.6 Постановка задачи.....	20
2.7 Результаты.....	21
3 Тестирование методом «Черный ящик».....	22
3.1 Теоретические сведения.....	22
3.1.1 Ключевые концепции и особенности тестирования чёрным ящиком.....	22
3.1.2 Основные техники тестирования чёрным ящиком.....	23
3.1.3 Эквивалентное разбиение.....	23
3.1.4 Анализ граничных значений.....	24
3.1.5 Таблицы решений (таблица переходов).....	24
3.1.6 Негативные сценарии.....	25
3.1.7 Тестирование по сценариям использования.....	25
3.2 Постановка задачи.....	26
3.3 Результаты.....	26
4 Модульное тестирование.....	27
4.1 Теоретические сведения.....	27
4.1.1 Основные понятия модульного тестирования.....	27
4.1.2 Основные черты модульного тестирования.....	27
4.1.3 Инструменты модульного тестирования.....	28
4.2 Модуль Qt Test.....	28
4.2.1 Создание модульного теста.....	28
4.2.1 Тестирование на основе данных.....	37
4.2.2 Моделирование событий GUI.....	39
4.2.3 Макросы QTest.....	42
4.4 Qt Quick Test.....	46
4.4.1 Создание модульного теста QtQuick.....	47
4.4.2 Тесты, основанные на данных.....	50
4.4.3 Контрольные показатели.....	51
4.4.4 Имитация событий клавиатуры и мыши.....	52

4.4.5 Другие типы QML для модульного тестирования.....	53
4.5 Постановка задачи.....	53
4.6 Результаты.....	53
5. Документирование программного кода.....	54
5.1 Специальные блоки комментариев.....	54
5.2 Оформление блока подробного описания.....	54
5.2.1 Первый вариант.....	54
5.2.2 Второй вариант.....	55
5.3 Оформление блока краткого описания.....	55
5.3.1 Первый вариант.....	56
5.3.2 Второй вариант.....	56
5.4 Однострочные комментарии для элементов.....	57
5.5 Пример документирования класса.....	57
5.6 Детальное описание метода класса.....	61
5.7 Оформление шапки кода.....	62
5.8 Структурные команды.....	63
5.9 Страницы документации.....	63
5.10 Документирование классов QML.....	64
5.10.1 Установка Doxyqml.....	64
5.10.2 Настройка Doxygen.....	64
5.10.3 Документирование типов. Функции.....	65
5.10.4 Свойства.....	65
5.10.5 Сигналы.....	66
5.10.6 Извлечение внутренних элементов.....	66
5.10.7 Пример документации QML-файла.....	66
5.10.8 Пояснения к тегам Doxyqml.....	70
5.11 Постановка задачи.....	72
5.12 Результаты.....	73
Рекомендуемая литература.....	73

Цель работы

Подготовка студентов к самостоятельному выполнению задач тестирования программного обеспечения и формированию необходимой документации.

Решаемые задачи

1. Разработка сценария и плана тестирования;
2. Проведение тестирования программного модуля по определенному сценарию. Проведение тестирования основанного на требованиях;
3. Проведение тестирования программного модуля методом «Черного ящика»;
4. Проведение тестирования программного модуля методом «Стеклянного ящика» (модульное тестирование);
5. Оформление баг- (дефект) репорт;
6. Исправление багов и ошибок, выявленных при тестировании;
7. Документирование программного кода модуля.

1 Теоретические сведения

1.1 Тестовая документация

- 1) **План тестирования** (*Test Plan*) — это отправная точка для всего процесса тестирования. В этом документе описано, что, как и когда будет проверяться. На основе этого плана строится дальнейшая тестовая документация и организуется сама проверка продукта.
- 2) **Тест-кейсы** (*Test Cases*) — это пошаговые сценарии для проверки конкретной функции, с чётким вводом и ожидаемым результатом.

- 3) **Набор тестов** (*Test Suite*) — группа связанных тест-кейсов, объединённых по одной теме или функции. Это удобно, чтобы запускать и проверять сразу несколько похожих сценариев.
- 4) **Тестовый сценарий** (*Test scenario*) - представляет собой пользовательский сценарий по тестированию определенной функциональности. Сценарий может иметь один или несколько Test Suite.
- 5) **Чек-листы** (*Checklists*) — простой список проверок без подробных инструкций. В нём отмечают, какие функции уже проверены и работают.
- 6) **Баг-репорты** (*Bug Reports*) — записи об ошибках с указанием, где и как их найти, что пошло не так и как воспроизвести проблему.

1.2 Тест-свит (набор тестов)

Тестовый набор — контейнер для выполнения тест-кейсов, сгруппированных по функциональности. В тест-план может входить много тестовых наборов, которые в свою очередь состоят из тест-кейсов.

Тест-кейсы выполняются вместе (последовательно); они группируются в наборы по функциональности (предназначению), в порядке, изложенном в тест-плане. Обычно постусловие одного тест-кейса используется в качестве предусловия для последующего. Тестовый набор может иметь статусы Активный, В процессе, и Завершен.

Таким образом, тестовый набор это коллекция тест-кейсов, направленных на проверку функциональности приложения, или какой-то его части.

В наборе также содержится информация о цели каждого тест-кейса, и конфигурация выполнения.

1.2.1 Примеры содержания наборов тест кейсов

- 1) Тест-кейсы для функционального тестирования страницы регистрации:
 - Обязательные поля ввода и инструкции;
 - Валидация полей;
 - Обработка ошибок;
 - Верификация электронной почты;
 - Медиа;
 - Условия предоставления услуг и политика конфиденциальности;

- Предотвращение дублирования учетных записей.
- 2) Тест-кейсы для тестирования безопасности страницы регистрации:
- Валидация ввода;
 - Защита паролей;
 - Управление сессиями и авторизация;
 - Конфиденциальность и защита данных;
 - Восстановление и верификация аккаунта.

1.2.2 Содержание тест-кейсов в тест-свитах

- 1) Обязательные поля ввода и инструкции
- Проверьте наличие всех обязательных полей на странице регистрации.
 - Убедитесь, что предоставлены понятные инструкции по заполнению формы регистрации.
 - Протестируйте поведение страницы при попытке отправить форму только с заполненным полем электронной почты.
 - Протестируйте, как ведет себя страница при попытке отправить форму с заполненным только полем пароля.
 - Убедитесь, что при попытке отправить форму с недостающей информацией для каждого пустого обязательного поля отображаются сообщения об ошибке, и проверьте их корректность.
 - Проверьте, работает ли на странице встроенная валидация каждого поля. По мере заполнения пользователем полей ввода должны появляться подсказки, которые показывают, являются ли введенные данные допустимыми.
 - Проверьте поведение страницы при отправке формы с корректной информацией во всех обязательных полях.
 - Убедитесь, что страница регистрации не позволяет отправить заявку, если какое-либо обязательное поле будет пустым.
 - Убедитесь, что страница регистрации сохраняет введенные данные даже при попытке отправки формы с некоторыми незаполненными обязательными полями, чтобы пользователям не пришлось заново вводить уже заполненные данные.
 - Проверьте, что не допускается использование пробелов в начале и в конце обязательных полей (например, имя, электронная почта).

- Протестируйте кейс, когда в поле электронной почты выводится пример адреса электронной почты либо другой текст, и убедитесь, что поле очищается, когда пользователь начинает набирать свой текст.
- Удостоверьтесь, что рядом с обязательными полями имеются всплывающие подсказки (тултипы) или значки помощи, которые объясняют особые условия или ограничения для вводимой информации.

2) Валидация полей

- Протестируйте отправку формы с незаполненными полями или полями, заполненными пробелами. Каждое поле отдельно.
- Проверьте ответ на отправку формы со всеми пустыми полями.
- Проверьте валидацию формата электронной почты и ответ на невалидные адреса электронной почты.
- Проверьте соблюдение критериев надежности пароля (длина, специальные символы, цифры и т. д.).
- Проверьте соответствие поля пароля и поля подтверждения пароля.
- Убедитесь, что корректно определяются и обрабатываются адреса электронной почты с несколькими символами “@” (например, “user@domain@domain.com”).
- Протестируйте обработку очень длинных адресов электронной почты, проверив, что установлен разумный лимит символов и выводится сообщение об ошибке при его превышении.
- Проверьте ответ страницы на ввод пароля без буквенно-цифровых символов (например, “*****”), чтобы убедиться, что страница обязательно требует пароль, содержащий буквы и цифры.
- Убедитесь, что корректно обрабатываются пароли с пробелами, спецсимволами и что они валидируются в соответствии с требуемыми критериями.
- Проверьте поведение, когда в полях ввода пароля и его подтверждения есть пробелы вначале или в конце, и убедитесь, что они корректно обрабатываются и обрезаются.
- Убедитесь, что страница регистрации распознает и не дает зарегистрировать часто встречающиеся или легко угадываемые пароли (например, “password”, “123456”, “admin”).

- Проверьте ответ страницы, когда поля для ввода пароля и подтверждения пароля содержат длинные строки с одинаковыми символами (например, “aaaaaaa”).
- Пароль, содержащий только символы “а”. Форма выдает сообщение об ошибке

1.3 Тест-кейс (тестовый случай)

Тест-кейс (Test Case) — это детально прописанная инструкция, предназначенная для проверки конкретной функции или аспекта программного продукта. Это основной инструмент тестирования, который определяет порядок шагов и ожидаемое поведение системы.

1.3.1 Содержание тест-кейса

- Идентификатор тестового кейса (Test Case ID): Идентификатор самого кейса;
- Заголовок кейса (Test Case Summary): Краткое и емкое название проводимой проверки;
- Связанное требование (Related Requirement): Идентификатор требования, к которому относится / отслеживается данный тестовый пример;
- Предварительные условия (Prerequisites): Любые предпосылки или предварительные условия, которые должны быть выполнены перед выполнением теста;
- Шаги выполнения (Test Script / Procedure): Шаги выполнения теста;
- Тестовые данные (Test Data): Тестовые данные или ссылки на тестовые данные, которые должны использоваться при проведении теста;
- Ожидаемый результат (Expected Result): результат, который мы ожидаем получить после выполнения шагов теста;
- Статус пройден или не пройден (Status): Другие статусы могут быть «Не выполнено», если тестирование не проводится, и «Заблокировано», если тестирование заблокировано;
- Заметки (Remarks): Любые комментарии к тесту или выполнению теста;
- Создано (Created By): Имя автора тестового примера;

- Дата создания (Date of Creation): Дата создания тестового примера (опционально модификации);
- Выполнено (Executed By): Имя человека, выполнившего тест;
- Дата выполнения (Date of Execution): Дата выполнения теста;
- Тестовое окружение (Test Environment): оборудование / программное обеспечение / сеть, в которых выполнялся тест, т.е. все необходимые сведения об окружении, чтобы можно было воспроизвести полученный результат.

1.3.2 Примеры Тест-кейсов

ID: TC_001

Назначение: Проверка успешной регистрации нового пользователя.

Предварительные условия: Пользователь не зарегистрирован в системе.

Шаг 1: Открыть страницу регистрации.

Шаг 2: Ввести уникальный email адрес.

Шаг 3: Ввести пароль длиной минимум 8 символов.

Шаг 4: Повторить пароль правильно.

Шаг 5: Нажать кнопку 'Зарегистрироваться'.

Ожидаемый результат: Появляется сообщение "Вы зарегистрированы успешно!".

Фактический результат: Сообщение появилось.

Результат: Прошел успешно.

ID: TC_002

Назначение: Проверка добавления товара в корзину.

Предварительные условия: Открыта страница товара.

Шаг 1: Найти интересующий товар на сайте магазина.

Шаг 2: Нажать кнопку "Добавить в корзину".

Шаг 3: Перейти в корзину покупок.

Ожидаемый результат: Товар находится в корзине покупок.

Фактический результат: Товар действительно присутствует в корзине.

Результат: Прошел успешно.

1.3.3 Статусы прохождения тест-кейсов

- «Пройден» (Passed). Означает, что ПО проверено и удовлетворило ожидания, работает исправно и соответствует требованиям.
- «Провален» (Failed). Поведение тестируемой системы не отвечает ожидаемым результатам, обнаружен дефект.
- «Заблокирован» (Blocked). Говорит о невозможности выполнения тестирования, существуют препятствия для проверки.
- «Пропущенный» (Skipped). Означает, что тестирование пропущено. Среди потенциальных причин — отсутствие нужного модуля для планируемой проверки.
- «Черновик» (Draft). Статус показывает, что либо отсутствует субъект проверки, либо она ещё не начиналась.
- «В процессе» (In progress). Указывает на долгосрочное исполнение.
- «Готов». Тест-кейсом можно пользоваться, он полностью актуален.
- «Ревью». Тест-кейс находится на проверке и ещё не готов к использованию.
- «В архиве». Тест-кейс уже не актуален, но оставлен для истории, чтобы можно было вернуться к нему за информацией.
- «Сломан». Тест-кейс не воспроизводится, или что-то пошло не так, и его нужно доработать.

1.3.4 Обозначение приоритетов

Приоритет тест-кейса может обозначаться разными способами, например: буквами (A, B, C, D, E), цифрами (1, 2, 3, 4, 5) или словами («крайне высокий», «высокий», «средний», «низкий», «крайне низкий»).

Часто используется такая градация приоритетов:

- Критический (Critical/P0) — проблемы, которые блокируют функциональность и влияют на базовые возможности.
- Высокий (High/P1) — серьёзное влияние на ключевые функции.
- Средний (Medium/P2) — заметные, но не критичные проблемы.
- Низкий (Low/P3) — минимальное влияние на пользовательский опыт.

При работе в команде может быть полезна единая система приоритетов, например, P1–P4 или Critical/High/Medium/Low.

1.3.5 Дополнительные элементы

- Тестовые данные — конкретные значения, используемые при тестировании.
- Зависимости — связи с другими тест-кейсами или требованиями
- Приложения — скриншоты, файлы данных

При составлении тест-кейсов важно соблюдать баланс между подробностью и лаконичностью. Согласно исследованию IEEE, оптимальный тест-кейс должен содержать от 3 до 8 шагов — такие кейсы выполняются на 42% быстрее и имеют на 27% меньше ошибок интерпретации, чем более длинные аналоги.

1.3.6 Шаблоны тест-кейсов для различных типов тестирования

Разные типы тестирования требуют разных подходов к составлению тест-кейсов. Рассмотрим специфику шаблонов для основных видов тестирования, с которыми сталкивается тестировщик.

- 1) **Функциональное тестирование.** Шаблон для функционального тестирования фокусируется на проверке соответствия функциональности требованиям:
 - ID: TCFUNC[номер]
 - Название: Проверка [конкретной функции]
 - Требование: [ссылка на требование]
 - Предусловия: [необходимое состояние системы]
 - Шаги: 1... 2... 3...
 - Ожидаемый результат: [детальное описание корректного поведения]
- 2) **Тестирование пользовательского интерфейса (UI).** Шаблон для UI-тестирования уделяет внимание визуальным аспектам и взаимодействию:
 - ID: TCUI[номер]
 - Название: Проверка отображения/поведения [элемента интерфейса]
 - Предусловия: [исходное состояние интерфейса]
 - Шаги: 1... 2... 3...

- Ожидаемый результат: [описание корректного отображения/поведения]
 - Визуальный эталон: [ссылка на макет/скриншот]
- 3) **API-тестирование.** Шаблон для API-тестирования содержит технические детали запросов и ответов:
- ID: TC-API[номер]
 - Название: Проверка [название метода API]
 - Endpoint: [URL-эндпоинта]
 - Метод: GET/POST/PUT/DELETE
 - Заголовки: [необходимые HTTP-заголовки]
 - Тело запроса: [JSON/XML структура]
 - Ожидаемый код ответа: [HTTP-код]
 - Ожидаемое тело ответа: [ожидаемая структура ответа]
- 4) **Тестирование безопасности.** Шаблон для тестирования безопасности фокусируется на проверке защищенности:
- ID: TC-SEC[номер]
 - Название: Проверка защиты от [тип уязвимости]
 - Тип уязвимости: [OWASP категория]
 - Предусловия: [исходное состояние системы]
 - Шаги: 1... 2... 3...
 - Вектор атаки: [детали проверки]
 - Ожидаемый результат: [описание корректного поведения системы при попытке атаки]

1.3.7 Высокоуровневый и низкоуровневый тест-кейсы

Высокоуровневый тест-кейс (*high level test case* или *logical test case*) - тест-кейс без конкретных входных данных и ожидаемых результатов. Как правило, ограничивается общими идеями и операциями, схож по своей сути с подробно описанным пунктом чек-листа. Достаточно часто встречается в интеграционном тестировании и системном тестировании, а также на уровне smoke. Может служить отправной точкой для проведения исследовательского тестирования или для создания низкоуровневых тест-кейсов.

Низкоуровневый тест-кейс (*low level test case*) - тест-кейс с конкретными входными данными и ожидаемыми результатами. Представляет собой «полностью готовый к выполнению» тест-кейс и вообще является наиболее классическим видом тест-кейсов. Начинающих тестировщиков чаще

всего учат писать именно такие тесты, т.к. прописать все данные подробно - намного проще, чем понять, какой информацией можно пренебречь, при этом не снизив ценность тест-кейса.

1.4 Чек-лист

Чек-лист (от англ. *checklist* — «список для проверки») — это структурированный перечень пунктов, который помогает выполнять действия или проверять состояние объекта, не упуская важных моментов.

Главная цель — обеспечить порядок и контроль в процессе работы, снизить вероятность ошибок и повысить эффективность.

Таблица 1 - Отличие чек-листа от тест-кейса

Чек-лист	Тест-кейс
Краткий список того, что проверить	Детальный сценарий с шагами и ожидаемыми результатами
Нет жёсткой последовательности действий	Чёткая последовательность шагов
Не содержит входных данных и ожидаемых результатов	Включает входные данные и ожидаемый результат
Подходит для быстрой проверки	Подходит для детального тестирования

1.4.1 Виды чек-листов

1) Read-Do («прочитай и сделай»)

- Пошаговая инструкция для выполнения процесса.
- Подходит для сложных процедур, где важно соблюдать порядок.
- Пример: предполётная проверка самолёта.

2) Do-Confirm («сделай и проверь»)

- Используется для самоконтроля после выполнения задачи.
- Содержит пункты для подтверждения результата.
- Пример: проверка готового продукта перед отправкой клиенту.

1.4.2 Пример чек-листа

Чек-лист проверки формы обратной связи

1. Поле «Имя»:

- принимает текст;
 - ограничивает длину (макс. 50 символов);
 - обязательное для заполнения.
2. Поле «Email»:
- проверяет формат (user@domain.com);
 - обязательное для заполнения.
3. Поле «Сообщение»:
- принимает текст (макс. 1000 символов);
 - не обязательное.
4. Кнопка «Отправить»:
- активна при заполнении обязательных полей;
 - отправляет данные при клике.
5. Сообщения об ошибках:
- появляются при неверном формате email;
 - отображаются при пустых обязательных полях.

1.5 Баг-репорт

Баг-репорт (*bug report*) — это официальная форма фиксации информации о найденной ошибке (дефекте) в программе или приложении. Грамотно оформленный баг-репорт облегчает быстрое восстановление ошибки инженерами-разработчиками.

Правильно подготовленный баг-репорт состоит из чёткого набора обязательных элементов, позволяющих быстро диагностировать проблему и исправить её.

1.5.1 Формат баг-репорта

- Идентификатор (ID): Уникальный номер, присваиваемый каждому багу.
- Заголовок (Summary/TITLE): Лаконичное и точное описание проблемы. Заголовок должен сразу давать ясное представление о характере ошибки.
- Проект (Project): Указывается проект или модуль, где была найдена ошибка.
- Версия продукта (Product Version): Версия продукта, на которой обнаружен баг.

- Платформа (Platform/Browsers): Информация о платформе, операционной системе, браузере и версии ОС, где воспроизводится проблема.
- Среда (Environment): Любая дополнительная информация о среде, где произошла ошибка (например, локализация, настройки сервера и т.д.).
- Критичность (Severity/Priority): Степень серьёзности ошибки. Может обозначаться такими категориями, как Blocker (блокирующая), Critical (критическая), Major (важная), Minor (средней важности), Trivial (несущественная).
- Тип ошибки (Type of Defect): Характер ошибки (например, логика программы, интерфейс, безопасность, производительность).
- Описание (Description): Подробное разъяснение проблемы. Желательно избегать технических жаргонов и писать понятно даже для начинающих инженеров.
- Шаги для воспроизведения (Steps to Reproduce): Пошаговые инструкции, следуя которым разработчики смогут повторить ошибку самостоятельно.
- Ожидаемый результат (Expected Result): Какой результат ожидается в нормальной ситуации.
- Фактический результат (Actual Result): То, что произошло фактически.
- Прикреплённые файлы (Attachments): Скриншоты, лог-файлы, видеозаписи и любые другие вспомогательные материалы, подтверждающие ошибку.
- Дополнительные комментарии (Additional Information): Любая полезная информация, уточняющая контекст проблемы или облегчающая диагностику.

1.5.2 Пример заполнения баг-репорта

Таблица 2 - Баг-репорт

Параметр	Значение
ID	BUG-001
Summary	Невозможно сохранить профиль пользователя
Project	Social Network
Product Version	v1.2
Platform	Windows 10, Firefox 95
Environment	Локальная разработка, среда Docker

Severity	High
Type	Functional
Description	Во время сохранения профиля кнопки "Сохранить" неактивны и невозможно завершить сохранение данных.
Steps to Reproduce	1. Зарегистрируйте новый аккаунт. 2. Перейдите в меню редактирования профиля. 3. Измените любое поле профиля. 4. Попробуйте сохранить изменения нажатием кнопки "Сохранить".
Expected Result	Профиль сохраняется успешно, кнопка активна.
Actual Result	Сохранение не происходит, кнопка "Сохранить" остаётся серой и неактивной.
Attachments	Скриншот экрана
Additional Information	Проблема наблюдается также в Safari на macOS.

2 Тестирование требований

2.1 Теоретические сведения

Тестирование требований — процесс проверки задокументированных требований к системе на:

- непротиворечивость;
- полноту;
- однозначность интерпретации;
- реализуемость;
- тестируемость.

Проводится до начала разработки, чтобы выявить и исправить дефекты документации на ранней стадии.

Тестирование требований позволяет:

- снизить затраты — дешевле исправить документ, чем переписывать код;
- избежать недопонимания между заказчиками, аналитиками, разработчиками и тестировщиками;
- сократить сроки разработки за счёт уменьшения итераций по уточнению требований;
- повысить качество продукта — чёткие требования снижают риск выпуска ПО, не соответствующего ожиданиям;
- упростить тестирование — на базе качественных требований легче создавать тест-кейс

2.2 Критерии качества требований

Корректность — требование описывает именно то, что должна делать система.

Недвусмысленность — формулировки одинаково интерпретируются всеми участниками.

Полнота — содержит всю необходимую информацию для понимания.

Непротиворечивость — нет конфликтов между требованиями.

Тестируемость — можно проверить выполнение требования.

Трассируемость — есть связь с бизнес-целями и другими требованиями.

Изменяемость — документ удобно обновлять.

Атомарность — требование нельзя разбить на более мелкие без потери смысла.

Выполнимость — реализация возможна в рамках проекта.

Актуальность — требования не устарели.

2.3 Этапы тестирования требований

1) Проверка структуры документа

- наличие всех разделов;
- соответствие утверждённому шаблону.

2) Анализ полноты

- учтены ли все пользователи/роли;
- описаны ли все сущности и их состояния;
- есть ли требования к взаимодействию с внешними системами;
- предусмотрены ли негативные сценарии и граничные условия.

3) Проверка непротиворечивости

- нет ли конфликтов между требованиями;
- согласуются ли текстовые описания с макетами и схемами.

4) Оценка однозначности

- отсутствуют ли двусмысленные термины;
- все ли понятия определены и используются единообразно.

5) Анализ тестируемости

- можно ли проверить выполнение требования;
- есть ли критерии приёмки.

6) Проверка трассируемости

- связь с бизнес-целями;
- соответствие функциональным и нефункциональным требованиям.

2.4 Техники тестирования требований

- Взаимный просмотр (peer review) — коллеги анализируют документ.
- Беглый просмотр — быстрая оценка структуры и ясности.

- Технический просмотр — анализ с точки зрения реализуемости.
- Формальная инспекция — структурированный процесс с фиксацией дефектов.
- Вопросы и ответы — выявление неясностей через диалог.
- Написание тест-кейсов — проверка, можно ли на основе требования создать тесты.
- Прототипирование — визуализация для уточнения требований.
- Таблицы решений — анализ сложных условий «если-то».
- Диаграммы и схемы — графическое представление логики.

2.5 Примеры типичных дефектов

- Неполнота: «Поле „Email“ обязательно» — нет проверки формата.
- Неоднозначность: «Система должна работать быстро» — нет метрики.
- Противоречие: в тексте сказано, что кнопка активна всегда, на макете — только при заполнении полей.
- Нетестируемость: «Интерфейс должен быть удобным» — нет критериев проверки.
- Отсутствие граничных условий: нет описания поведения при пустом вводе или слишком длинных строках.

2.6 Постановка задачи

Произвести тестирование предварительных требований курсового проекта и составить отчет о недостатках, найденных проблемах и путях их устранения.

Допустим, мы тестируем приложение для бронирования билетов на поезд, и одно из требований звучит так: Пользователь должен иметь возможность выбрать билет с минимальным количеством пересадок.

Таблица 3 - Пример заполненного тест-кейса для тестирования требования

Параметр	Значение
ID	TC-RB-001
Название	Проверка выбора билета с наименьшими пересадками
Описание	Убедимся, что приложение показывает билеты с минимальным количеством пересадок.

Предусловие	Пользователю доступно бронирование поездов.
Процедура	1. Откройте приложение. 2. Выберите направление и дату. 3. Настройте фильтр "Минимальное количество пересадок". 4. Посмотрите предложенный вариант маршрута.
Ожидаемый результат	Среди предложений появляются маршруты с наименьшим количеством пересадок.
Фактический результат	См. при выполнении теста.
Статус	Pass
Комментарии	Требование выполняется верно.
Автор	Иванов Иван
Дата последнего обновления	01.12.2025

В случае невозможности создания тест-кейса по элементу требований — это расценивается как дефект и фиксируется в отчетной таблице.

2.7 Результаты

Преподавателю предоставляется следующая отчетность по практике:

1. Тест-свит с набором первичных тест-кейсов;
2. Отчет оформленный в виде таблицы.

Пример оформления:

Таблица 4 — Отчет о проведении тестирования требований

ID	Тип недостатка	Рекомендуемые изменения	Ссылка на требования
1	Нет описания роли администратора. Как именно реализован доступ к управлению товарами?	Необходимо дополнить описанием полномочий и прав администратора.	ТР N
2	Одно требование гласит, что доступ к управлению доступен только администратору, другое предполагает, что покупатели могут управлять своими заказами. Нужно уточнить разделение ролей.	Следует разделить права покупателей и администраторов, добавив детализацию ролей и их полномочий.	ТР M

3 Тестирование методом «Черный ящик»

3.1 Теоретические сведения

Тестирование черным ящиком (*black-box testing*) — представляет собой методику тестирования программного обеспечения, при которой внутренний механизм функционирования программы неизвестен тестировщику. Вместо этого внимание уделяется поведению программы на основании входных данных и наблюдаемым результатам (выходным данным).

Основная идея тестирования чёрным ящиком заключается в следующем:

- Входные данные предоставляются в программу.
- Наблюдаются полученные выходные данные.
- Сравниваются ожидания с фактическим результатом.

Методология основана исключительно на понимании требований и спецификаций системы, без погружения в внутреннюю реализацию и коды.

3.1.1 Ключевые концепции и особенности тестирования чёрным ящиком

- Незнание внутренней структуры: Тестировщик рассматривает программу как чёрный ящик, не имея представления о внутреннем устройстве и структуре программы.
- Внешнее поведение: Акцент делается на поведении системы, то есть реакции на внешние воздействия и взаимодействия с пользователем.
- Моделирование пользовательского поведения: Тестирование направлено на симуляцию поведения пользователя и проверку того, соответствуют ли результаты поставленным целям.
- Преимущество простоты и универсальности: Поскольку методика не требует глубокого понимания внутренностей программы, её легко осваивать и применять независимо от сложности самой программы.

3.1.2 Основные техники тестирования чёрным ящиком

- **Эквивалентное разбиение** (*Partitioning*): Данные разбиваются на равнозначные подмножества, и выбираются представители из каждого подмножества для тестирования.
- **Анализ граничных значений** (*Boundary Value Analysis*): Особое внимание уделяют значимым переходам и краям диапазонов, чтобы выявить ошибки вблизи граничного значения.
- **Таблицы решений** (*Decision Tables*): Логические условия и их последствия сводятся в табличную форму для последующего тестирования.
- **Тестирование состояний и переходов** (*State Transition Testing*): Представляют систему как машину состояний и проводят тестирование на каждом переходе между ними.
- **Тестирование по сценариям использования** (*Use Case Testing*) — это техника тест-дизайна, при которой рассматривается взаимодействие системы и пользователя через призму конкретных сценариев. Сценарии описываются с точки зрения пользователя, а не системы.

3.1.3 Эквивалентное разбиение

Техника включает в себя разделение входных значений на допустимые и недопустимые разделы и выбор репрезентативных значений из каждого раздела в качестве тестовых данных.

Может быть использована для уменьшения количества тестовых случаев. Допустим, имеется целая переменная N в диапазоне от -99 до 99: позитивными классами эквивалентности будут [-99, -10], [-9, -1], 0, [1, 9], [10, 99], а недействительными (негативными) – < -99, > 99, пустое значение, нечисловые строки.

Например, поле «Возраст» должно валидироваться и принимать значения от 18 до 99 лет. Тесты можно разбить на 4 класса:

- Класс 1 (валидные): числа от 18 до 99, например 25 или 60.
- Класс 2 (невалидные): меньше 18, например 0 или 17.
- Класс 3 (невалидные): больше 99, например 120.
- Класс 4 (невалидные): строки, спецсимволы, отрицательные числа.

Т. е. нужно выполнить всего 4 тест-кейса

3.1.4 Анализ граничных значений

Техника, которая включает в себя определение границ входных значений и выбор в качестве тестовых данных значений, находящихся на границах, внутри и вне границ.

При проверке мы берем следующие величины: минимум, (минимум-1), максимум, (максимум+1), стандартные значения. Например, в том же случае $-99 \leq N \leq 99$ будет использоваться набор: -100, -99, -98, -10, -9 -1, 0, 1, 9, 10, 98, 99, 100.

3.1.5 Таблицы решений (таблица переходов)

Когда результат зависит от нескольких условий, обычные тесты не помогают. Нужна системная проверка комбинаций. Такая техника помогает быстро проверить сложную бизнес-логику и гарантировать, что система работает корректно.

Чтобы обнаружить дефекты, нужно «перемножить» все возможные состояния на все возможные действия. Пример таблицы переходов представлен на Рисунке 1.

Текущее состояние	Действие	Переход	Следующее состояние
Нет билета	1. Поиск билета	Поиск	Билет выбран
	2. Бронь билета		Null
	3. Покупка билета		Null
	4. Отмена брони		Null
	5. Выход с сайта	Выход	Выход
	6. Время истекло		Нет билета
Бронь билета	1. Поиск билета	Новый поиск?	Билет выбран?
	2. Бронь билета		Null
	3. Покупка билета	Покупка	Билет куплен
	4. Отмена брони	Отмена брони	Билет выбран
...	...	...	...

Рисунок 1 - Таблица переходов

3.1.6 Негативные сценарии

Цель этой техники обратная: не подтвердить работу системы, а попытаться её сломать. Тестировщик вводит некорректные данные и проверяет, как приложение реагирует.

Примеры:

- Ввести -5 в поле возраста.
- Подставить SQL-инъекцию ('or 1=1) в форму входа.
- Вставить JavaScript (<script>alert('xss')</script>) в комментарий.
- Отправить пустую форму при обязательных полях.

Если система выдаёт понятную ошибку и не падает — тест пройден. Если не выводит ошибку и останавливает работу или пропускает вредоносный код — это баг или уязвимость.

3.1.7 Тестирование по сценариям использования

Эта техника используется при написании тестов для индивидуального сценария пользователя с целью определить, как пользователи будут взаимодействовать с системой, чтобы достичь определённых целей.

Некоторые преимущества тестирования по сценариям использования:

- Формирование общего видения продукта. Сценарии дают представление о том, как система должна функционировать и какие задачи решать.
- Единое понимание функциональных требований. Use Case фокусируется на том, что должна делать система, не уходя в детали реализации.
- Ускорение подготовки тестов. Чётко сформулированные сценарии использования позволяют тестировщикам быстро составлять сценарии тестирования, экономя время на анализ требований.
- Отслеживание соответствия системы потребностям. Каждый Use Case привязан к определённой бизнес-цели или потребности пользователя.
- Основа для технической детализации. Use Case можно рассматривать как скелет, на который затем накладываются подробности реализации.

Пример пользовательского сценария для приложения, осуществляющего заказа пиццы:

- Пользователь заходит на главную страницу заказа.
- Пользователь выбирает пиццу из меню.
- Пользователь добавляет пиццу в корзину.

- Пользователь переходит в корзину и проверяет свой заказ.
- Пользователь вводит свои контактные данные и адрес доставки.
- Пользователь выбирает метод оплаты и подтверждает заказ.
- Пользователь получает уведомление об успешном оформлении заказа.

3.2 Постановка задачи

- 1) Разработать наборы тест-кейсов (тест-свит) для проведения тестирования методом «Черного ящика» (не менее 10 тест-кейсов).
Вместо тест-свита вы можете разработать тестовый сценарий (*Scenario testing*). Оформление аналогично оформлению тест-свита, но содержание должно подчиняться логике тестового сценария.
- 2) Выполнить тестирование с использованием данной методики и разработанных тест-кейсов.
- 3) Оформить баг-репорт.

3.3 Результаты

Преподавателю предоставляется следующая отчетность по практике:

- 1) Тест-свит/Тест-сценарий, с приложением артефактов (скриншоты окна программы, таблица переходов);
- 2) Баг-репорты.

4 Модульное тестирование

4.1 Теоретические сведения

Модульное тестирование (*unit testing*) — это процесс тестирования отдельных небольших единиц программного кода (обычно отдельные функции, модули или классы) отдельно от остальной части системы, с целью проверки корректности их работы. Главная цель модульного тестирования — изолированно проверить каждый компонент на предмет правильной реализации.

Идея модульного тестирования состоит в том, что параллельно основному компоненту программы, который включает непосредственно алгоритмы её работы, создаётся дополнительный «тестовый», в котором имитируется работа основного компонента в тех или иных условиях. По результатам выполнения «тестового» компонента судят о правильности работы основного.

4.1.1 Основные понятия модульного тестирования

- **Единица (unit)** — самый маленький элемент программного обеспечения, который можно подвергнуть отдельному тестированию. Чаще всего единицами являются функции, методы классов или небольшие модули.
- **Unit-тест** — короткий тестовый сценарий, предназначенный для проверки конкретной единицы программного кода.
- **Mock-объект** — специальный объект, имитирующий реальный объект для упрощения тестирования. Mock-объекты часто используются для изоляции тестируемого компонента от остальных частей системы.

4.1.2 Основные черты модульного тестирования

- **Изоляция:** модульные тесты запускают отдельные фрагменты кода отдельно от всей системы.
- **Быстрота:** поскольку проверяется лишь небольшая единица, тесты выполняются очень быстро.

- **Независимость:** каждый тест самостоятелен и не влияет на остальные тесты.
- **Минималистичность:** тесты фокусируются на конкретных функциях и возвращаемых ими результатах.

4.1.3 Инструменты модульного тестирования

Существует множество автоматизированных инструментов, которые позволяют разработчикам создавать и запускать тест-кейсы, а также проводить автоматический анализ результатов тестирования. Среда Qt поддерживает следующие фреймворки для тестирования:

- QTest — собственная среда тестирования Qt;
- GTest — система тестирования от Google;
- Boost.Test — библиотека для тестирования от проекта Boost
- Catch2 — фреймворк для тестирования на C++

4.2 Модуль Qt Test

Qt Test — это фреймворк для модульного тестирования приложений и библиотек на базе Qt. Qt Test предоставляет все функции, которые обычно есть во фреймворках для модульного тестирования, а также расширения для тестирования графических пользовательских интерфейсов. С помощью мастера Qt Creator можно создать проект, содержащий тесты Qt, а также собрать и запустить их непосредственно из Qt Creator.

4.2.1 Создание модульного теста

- Запустите среду разработки Qt Creator;
- Файл → New Project;
- Выбрать шаблон Test Project → Qt Test Project → Выбрать;
- Название (например) test_suite_1 → Далее;
- Имя теста test_case_1, Отметить: Требуется QApplication, Создать код инициализации и очистки, Выбрать: Система сборки CMake, Далее;
- Выбрать комплект, Далее, Завершить.

Шаги создания тестового проекта продемонстрированы на рисунках ниже.

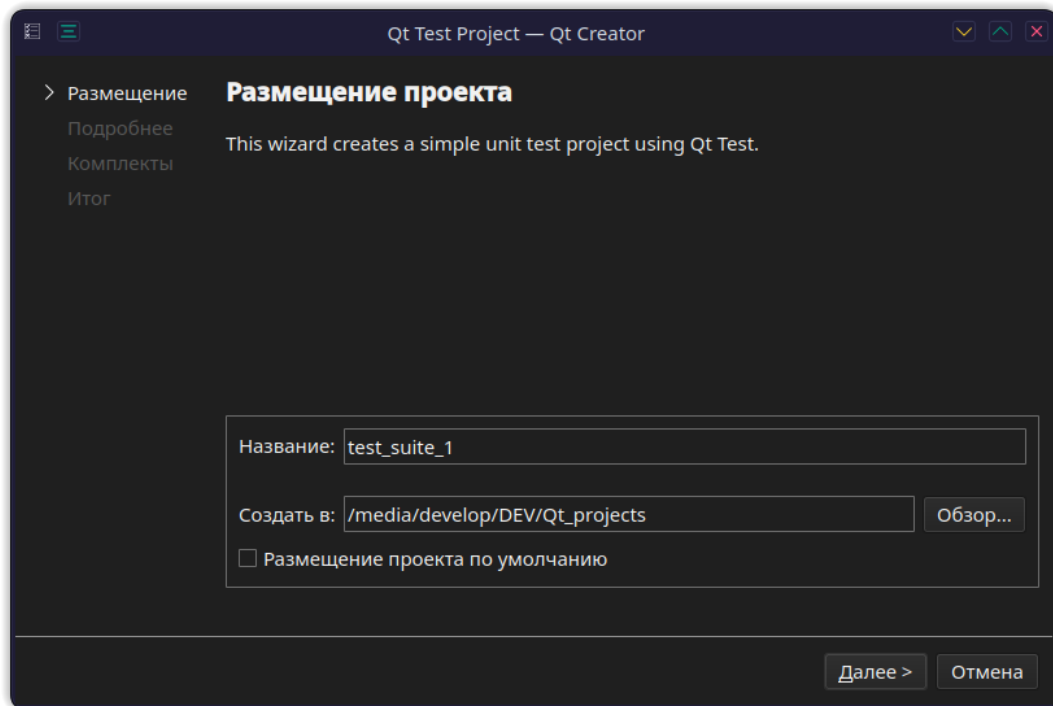


Рисунок 2

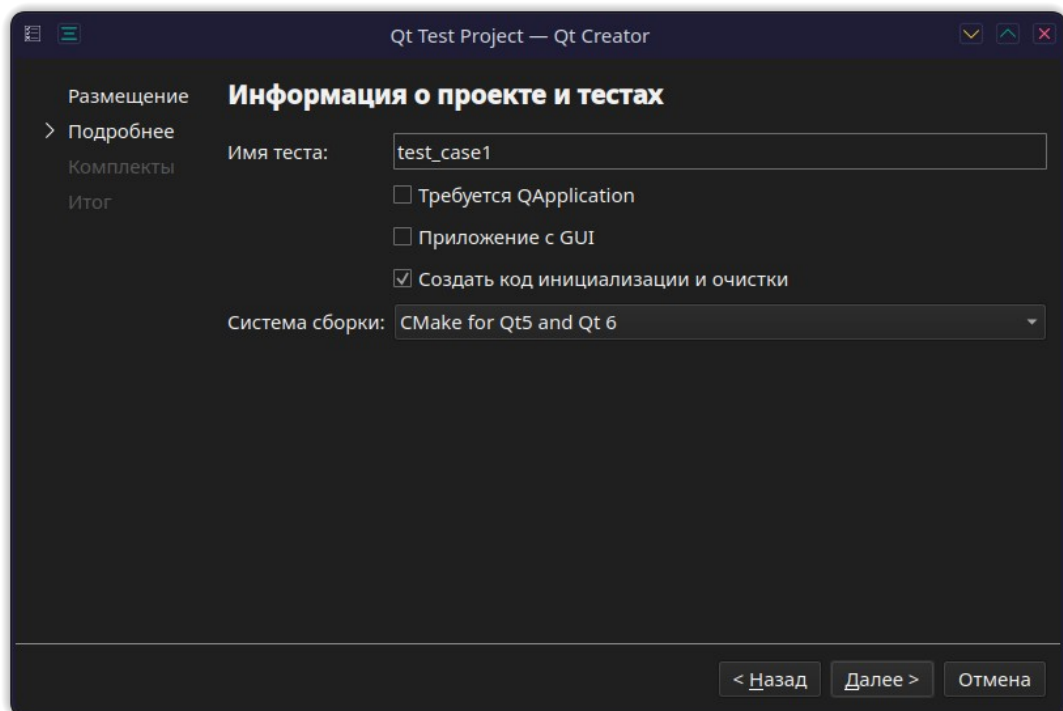


Рисунок 3

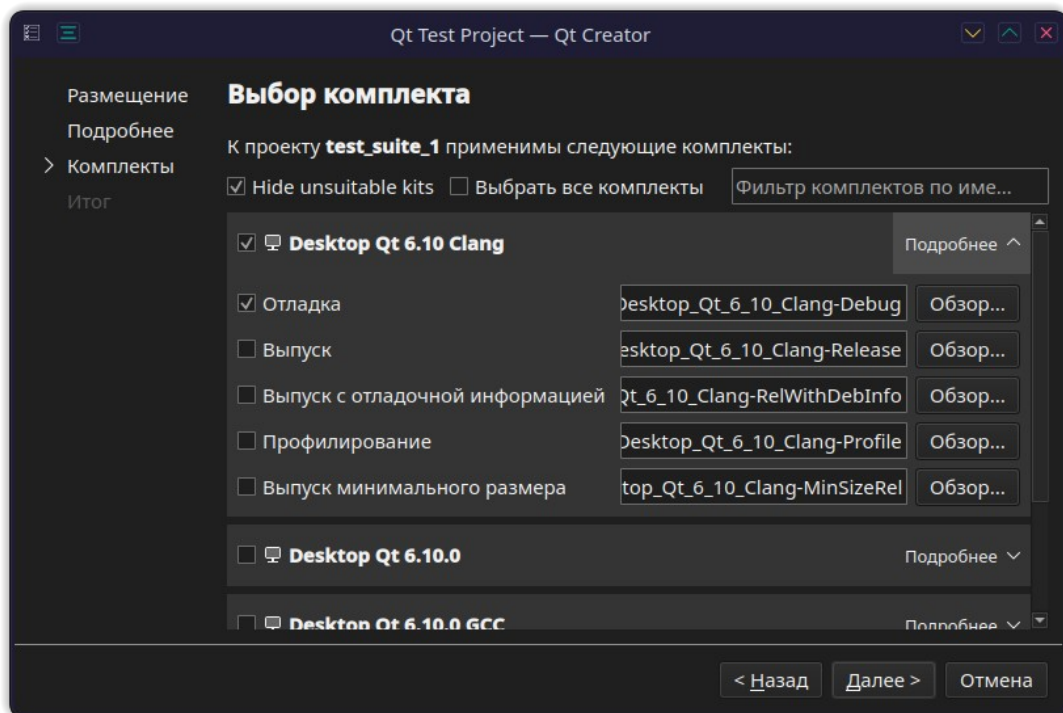


Рисунок 4

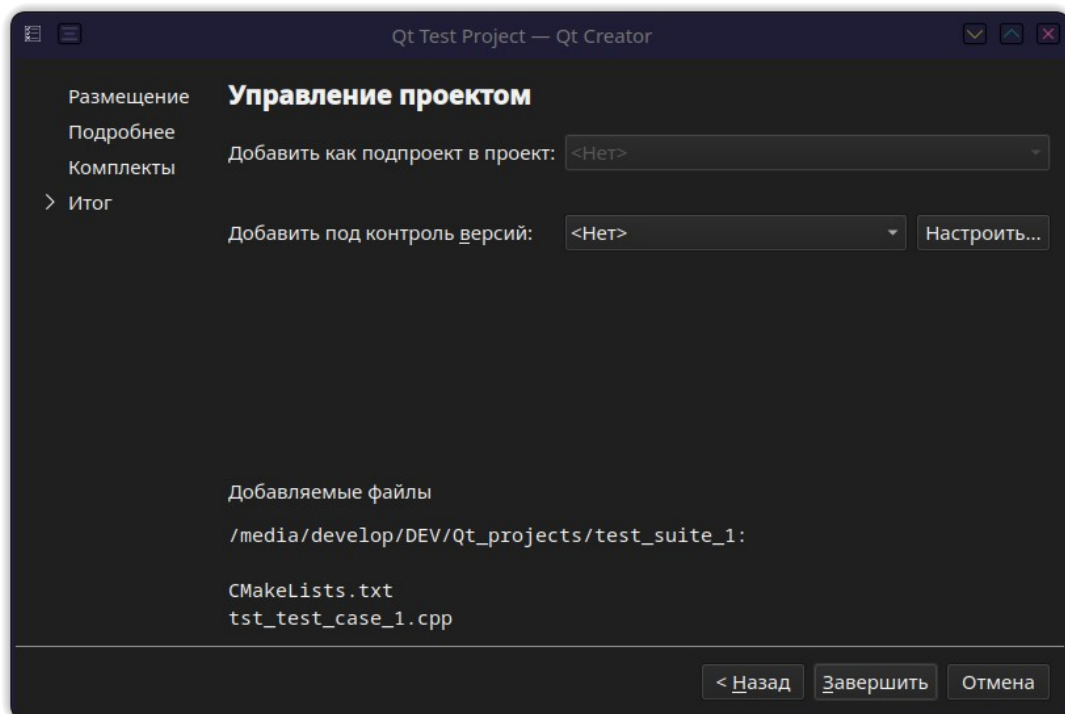


Рисунок 5

Чтобы создать тестовый класс, необходимо создайте подкласс `QObject` и добавьте в него один или несколько частных слотов. **Каждый частный слот — это тестовая функция в вашем тесте.**

Помимо собственных частных слотов, можно определить следующие частные слоты, которые не рассматриваются как тестовые функции. Они используются для инициализации и очистки всего теста или текущей тестовой функции:

- `initTestCase()` - будет вызван перед выполнением первой тестовой функции.
- `initTestCase_data()` - будет вызван для создания глобальной таблицы тестовых данных.
- `cleanupTestCase()` - будет вызван после выполнения последней тестовой функции.
- `Init()` - будет вызван перед выполнением каждой тестовой функции.
- `Cleanup()` - будет вызван после выполнения каждой тестовой функции.

Если вы выполнили все предписанные выше шаги, в файле `tst_test_suite_1.cpp` система сгенерирует следующий код:

Листинг 1

```
1.  #include <QTest>
2.
3.  // add necessary includes here
4.
5.  class test_suite_1 : public QObject
6.  {
7.      Q_OBJECT
8.
9.  public:
10.     test_suite_1();
11.     ~test_suite_1();
12.
13. private slots:
14.     void initTestCase();
15.     void init();
```

```

16.     void cleanupTestCase();
17.     void cleanup();
18.     void test_case1();
19. };
20.
21. test_suite_1::test_suite_1() {}
22.
23. test_suite_1::~~test_suite_1() {}
24.
25. void test_suite_1::initTestCase()
26. {
27.     // code to be executed before the first test function
28. }
29.
30. void test_suite_1::init()
31. {
32.     // code to be executed before each test function
33. }
34.
35. void test_suite_1::cleanupTestCase()
36. {
37.     // code to be executed after the last test function
38. }
39.
40. void test_suite_1::cleanup()
41. {
42.     // code to be executed after each test function
43. }
44.
45. void test_suite_1::test_case1() {}
46.
47. QTEST_APPLESS_MAIN(test_suite_1)
48.

```

```
49. #include "tst_test_suite_1.moc"
```

Специальный макрос QTEST_APPLESS_MAIN создает функцию main(), которая запускает все тестовые функции. Данная весия макроса (всего их - 3) не создает экземпляра объекта QApplication.

Поскольку объявление и реализация находятся в файле сpp, также необходимо подключить сгенерированный файл moc, для выполнения работы анализатора Qt.

Для начала тестирования необходимо создать класс, содержащий набор тестируемых функций и включить его в файл теста. Для простоты, в коде ниже, функции созданы вне класса.

Листинг 2 - Заголовочный файл тестируемых функций разработчика

```
1. #ifndef TEST_MYLIB_H
2. #define TEST_MYLIB_H
3.
4. int max(int, int);
5. int min(int, int);
6.
7. #endif // TEST_MYLIB_H
```

Листинг 3 - сpp-файл разработчика с тестируемыми функциями

```
1. #include "test_mylib.h"
2.
3. int max(int a, int b)
4. {
5.     return a > b ? a : b;
6. }
7.
8. int min(int a, int b)
9. {
10.    return a > b ? b : a;
11. }
```

Листинг 4 - Файл теста с двумя тест-кейсами

```
1.  #include <QTest>
2.  #include "test_mylib.h"
3.
4.  class test_suite_1 : public QObject
5.  {
6.      Q_OBJECT
7.
8.  public:
9.      test_suite_1();
10.     ~test_suite_1();
11.
12. private slots:
13.     void initTestCase();
14.     void init();
15.     void cleanupTestCase();
16.     void cleanup();
17.     void test_case1(); // Test Case #1 => проверка min
18.     void test_case2(); // Test Case #2 => проверка max
19. };
20.
21. test_suite_1::test_suite_1() {}
22. test_suite_1::~~test_suite_1() {}
23.
24. void test_suite_1::initTestCase()
25. {
26.     qDebug("Called before everything else.");
27. }
28.
29. void test_suite_1::init() {}
30.
31. void test_suite_1::cleanupTestCase()
32. {
```

```

33.     qDebug("Called after myFirstTest and mySecondTest.");
34. }
35.
36. void test_suite_1::cleanup() {}
37.
38. void test_suite_1::test_case1() {
39.     QCOMPARE(min(25, 0), 0);
40.     QCOMPARE(min(-12, -5), -12);
41.     QCOMPARE(min(2007, 2007), 2007);
42.     QCOMPARE(min(-12, 5), -12);
43. }
44.
45. void test_suite_1::test_case2() {
46.     QCOMPARE(max(25, 0), 25);
47.     QCOMPARE(max(-12, -5), -5);
48.     QCOMPARE(max(2007, 2007), 2007);
49.     QCOMPARE(max(-12, 5), 5);
50. }
51.
52. QTEST_APPLESS_MAIN(test_suite_1)
53.
54. #include "tst_test_suite_1.moc"

```

Запустите тестовый проект (Ctrl+R).

Вывод программы после запуска тестового проекта:

```

***** Start testing of test_suite_1 *****
Config: Using QtTest library 6.10.0, Qt 6.10.0 (x86_64-
little_endian-lp64 shared (dynamic) release build; by GCC 11.4.1
20231218 (Red Hat 11.4.1-3)), fedora 43
QDEBUG : test_suite_1::initTestCase() Called before everything
else.
PASS   : test_suite_1::initTestCase()
PASS   : test_suite_1::test_case1()
PASS   : test_suite_1::test_case2()

```

```
QDEBUG : test_suite_1::cleanupTestCase() Called after
myFirstTest and mySecondTest.
PASS    : test_suite_1::cleanupTestCase()
Totals: 4 passed, 0 failed, 0 skipped, 0 blacklisted, 0ms
***** Finished testing of test_suite_1 *****
```

Для запуска тестов используем панель Tests в QtCreator (в сайд-баре справа), где отображаются все юнит-тесты, которые среда разработки распознала в нашем проекте.

Через контекстное меню этой панели можно запустить как все тесты разом, так и тесты отдельного класса, либо отдельный тест-метод. Есть возможность запуска тест-кейсов под отладчиком.

Для работы с результатами выполнения теста, а также их перезапуска, используется вкладка нижней панели Test Results (Рисунок 6).

Если юнит-тесты "зеленые", то они пройдены успешно, в противном случае, тесты (все или отдельные) не пройдены.

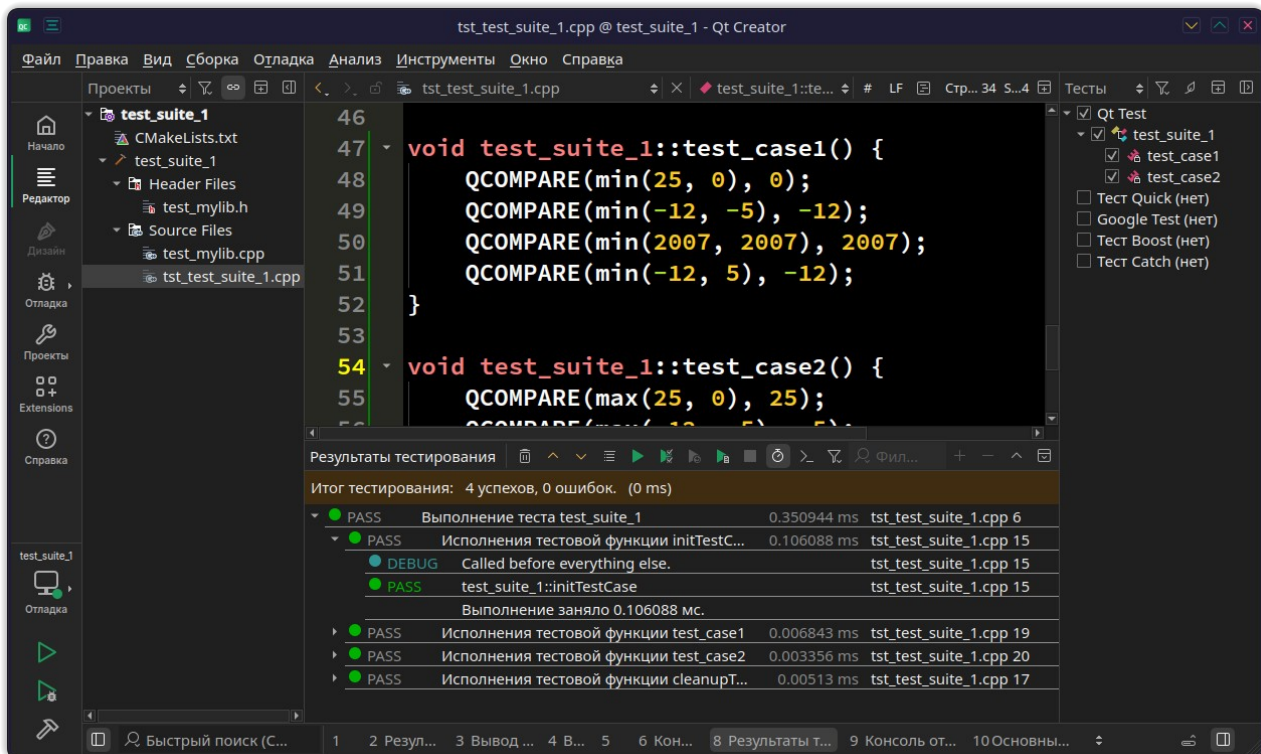


Рисунок 6

4.2.1 Тестирование на основе данных

В листинге 4 отдельные шаги тест-кейса мы программировали соответствующими вызовами через макрос QCOMPARE. Если мы добавим больше тестовых данных, то код будет сильно перегружен.

Чтобы функция не была перегружена повторяющимся кодом, Qt Test поддерживает добавление в тестовую функцию тестовых данных. Всё, что нам нужно, — это добавить в наш тестовый класс ещё один приватный слот.

Листинг 5

```
1.  #include <QTest>
2.  #include "test_mylib.h"
3.
4.  class test_suite_1 : public QObject
5.  {
6.      Q_OBJECT
7.
8.  public:
9.      test_suite_1();
10.     ~test_suite_1();
11.
12. private slots:
13.     void initTestCase();
14.     void init();
15.     void cleanupTestCase();
16.     void cleanup();
17.     void test_case1(); // Test Case #1 => проверка min
18.     void test_case2(); // Test Case #1 => проверка max
19.     void test_case1_data(); // Данные для Test Case #1
20. };
21.
22. // ...
23.
24. void test_suite_1::test_case1_data()
```

```

25. {
26.     QTest::addColumn<int>("a");
27.     QTest::addColumn<int>("b");
28.     QTest::addColumn<int>("result");
29.
30.     QTest::newRow("25 и 0") << 25 << 0 << 0;
31.     QTest::newRow("-12 и -5") << -12 << -5 << -12;
32.     QTest::newRow("2007 и 2007") << 2007 << 2007 << 2007;
33.     QTest::newRow("-12 и 5") << -12 << 5 << -12;
34. }
35.
36. void test_suite_1::test_case1() {
37.     QFETCH(int, a);
38.     QFETCH(int, b);
39.     QFETCH(int, result);
40.     QCOMPARE(min(a, b), result);
41. }
42.
43. // ...
44.
45. QTEST_APPLESS_MAIN(test_suite_1)
46. #include "tst_test_suite_1.moc"

```

Сначала мы определяем три элемента нашей тестовой таблицы с помощью функции `QTest::addColumn()`: аргументы `a` и `b`, а также ожидаемый результат применения функции `result`. Затем мы добавляем в таблицу некоторые данные с помощью функции `QTest::newRow()`. `QTest::newRow()` принимает один аргумент: имя, которое будет связано с набором данных и использоваться в журнале тестирования для идентификации строки данных. `QTest::addRow()` принимает строку формата (в стиле `printf`), за которой следуют параметры, которые будут представлены вместо токенов форматирования в строке формата. Затем мы передаём набор данных в новую строку таблицы.

Функция `min()`, будет выполнена столько раз, сколько строк данных было добавлено в таблицу данных.

4.2.2 Моделирование событий GUI

Qt Test есть несколько механизмов для тестирования графических пользовательских интерфейсов. Вместо имитации событий собственной оконной системы Qt Test отправляет внутренние события Qt.

Для создания тестового проекта с GUI вам необходимо выполнить все шаги, которые описаны выше. В окне мастера создания проектов «Информация о проекте и тестах» необходимо выбрать все компоненты, как показано на рисунке 7 ниже:

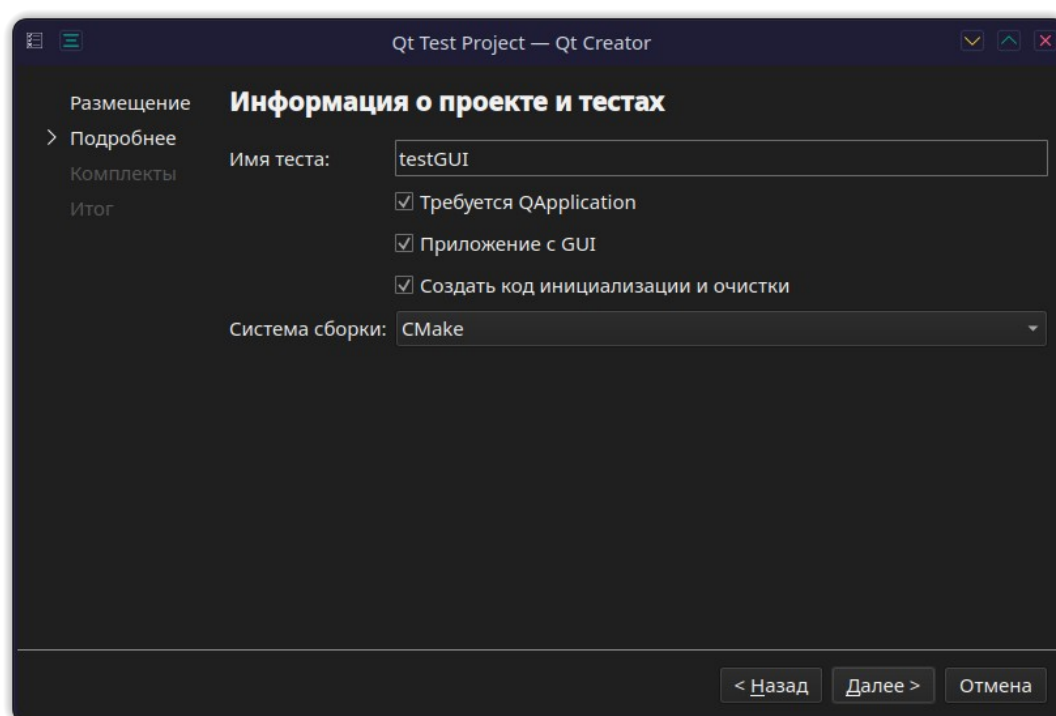


Рисунок 7

Предположим, что вы хотите протестировать поведение нашего класса `QLineEdit`. Как и прежде, вам понадобится класс, содержащий тестовую функцию. Разница лишь в том, что вам нужно добавить определения классов графического интерфейса Qt в дополнение к пространству имён `QTest`.

При реализации тестовой функции сначала мы создаём `QLineEdit`. Затем с помощью функции `QTest::keyClicks()` имитируем запись «hello world» в строке редактирования.

`QTest::keyClicks()` имитирует нажатие последовательности клавиш на виджете. При желании можно указать модификатор клавиатуры, а также задержку (в миллисекундах) теста после каждого нажатия клавиши.

Аналогичным образом можно использовать функции `QTest::keyClick()`, `QTest::keyPress()`, `QTest::keyRelease()`, `QTest::mouseClick()`, `QTest::mouseDClick()`, `QTest::mouseMove()`, `QTest::mousePress()`, `QTest::mouseRelease()` для имитации связанных событий GUI.

Наконец, мы используем макрос `QCOMPARE()`, чтобы проверить, соответствует ли текст редактирования строки ожидаемому.

Листинг 6

```
1.  #include <QCoreApplication>
2.  #include <QTest>
3.  #include <QLineEdit>
4.
5.  class testGui : public QObject
6.  {
7.      Q_OBJECT
8.
9.  public:
10.     testGui();
11.     ~testGui();
12.
13. private slots:
14.     void initTestCase();
15.     void init();
16.     void cleanupTestCase();
17.     void cleanup();
18.     void test_case1();
19. };
20.
21. //...
22.
23. void testGui::test_case1()
24. {
25.     QLineEdit lineEdit;
26.     QTest::keyClicks(&lineEdit, "hello world");
```

```

27.     QCOMPARE(lineEdit.text(), QString("hello world"));
28. }
29.
30. QTEST_MAIN(testGui)
31. #include "tst_testgui.moc"

```

Листинг 7 - Настройка параметров сборки в файле CMakeLists.txt

```

1.  cmake_minimum_required(VERSION 3.21)
2.
3.  project(test_suite_gui_2 VERSION 0.1 LANGUAGES CXX)
4.
5.  enable_testing()
6.
7.  set(CMAKE_CXX_STANDARD 23)
8.  set(CMAKE_CXX_STANDARD_REQUIRED ON)
9.
10. find_package(Qt6 REQUIRED COMPONENTS Core Gui Test Widgets)
11.
12. qt_standard_project_setup()
13.
14. qt_add_executable(testGui
15.     tst_testgui.cpp
16. )
17. add_test(NAME testGui COMMAND testGui)
18.
19. set_target_properties(testGui PROPERTIES
20.     MACOSX_BUNDLE FALSE
21.     WIN32_EXECUTABLE FALSE
22. )
23.
24. target_link_libraries(testGui
25.     PRIVATE Qt6::Core
26.     PRIVATE Qt6::Gui

```

```
27.     PRIVATE Qt6::Test
28.     PRIVATE Qt6::Widgets
29. )
```

Вывод программы:

```
***** Start testing of testGui *****
Config: Using QTest library 6.10.0, Qt 6.10.0 (x86_64-
little_endian-lp64 shared (dynamic) release build; by GCC 11.4.1
20231218 (Red Hat 11.4.1-3)), fedora 43
PASS   : testGui::initTestCase()
PASS   : testGui::test_case1()
PASS   : testGui::cleanupTestCase()
Totals: 3 passed, 0 failed, 0 skipped, 0 blacklisted, 17ms
***** Finished testing of testGui *****
```

Как создать повторное воспроизведение событий для GUI-теста описано на странице документации здесь: <https://doc.qt.io/qt-6/qtestlib-tutorial4-example.html>

4.2.3 Макросы QTest

- QBENCHMARK - Этот макрос используется для измерения производительности кода в рамках теста. Код, который необходимо протестировать, содержится в блоке кода после этого макроса.

```
void TestBenchmark::simple()
{
    QString str1 = QLatin1String("This is a test string");
    QString str2 = QLatin1String("This is a test string");
    QCOMPARE(str1.localeAwareCompare(str2), 0);
    QBENCHMARK {
        str1.localeAwareCompare(str2);
    }
}
```

- COMPARE(actual, expected) - Макрос QCOMPARE() сравнивает значение actual со значением expected с помощью оператора равенства. Если actual

и expected совпадают, выполнение продолжается. Если нет, в журнал тестирования записывается сообщение об ошибке, и функция тестирования возвращается без выполнения последующих проверок.

Необходимо соблюдать семантику параметров QCOMPARE(). Первым параметром всегда должно быть фактическое значение, полученное в результате выполнения тестируемого кода, а вторым параметром всегда должно быть ожидаемое значение. Если значения не совпадают, QCOMPARE() выводит их с пометками Actual и Exected. Если порядок параметров поменять местами, отладка неудачного теста может вызвать затруднения, а тесты, ожидающие нулевое значение, могут завершиться ошибкой из-за ошибок округления.

QCOMPARE() пытается вывести содержимое значений, если сравнение не удалось, чтобы в журнале тестирования было видно, почему сравнение не удалось.

```
QCOMPARE(QString("hello").toUpper(), QString("HELLO"));
```

- QFAIL(message) - тот макрос можно использовать для принудительного сбоя теста. Тест останавливается, и в журнал тестирования добавляется сообщение о сбое message

```
if (sizeof(int) != 4)
    QFAIL("This test has not been ported to this platform
yet.");
```

- QFETCH(type, name) - Макрос выборки создает локальную переменную с именем name и с типом type в стеке. name и type должны соответствовать столбцу из таблицы данных теста. Это утверждение, и тест будет прерван, если утверждение завершится неудачей.
- QSKIP(description) - Если макрос QSKIP() вызывается из тестовой функции, он останавливает выполнение теста, не добавляя сообщение об ошибке в журнал тестирования. С его помощью можно пропускать тесты, которые не имеют смысла в текущей конфигурации. Например, тест на отображение шрифтов может вызывать QSKIP(), если в тестовой системе не установлены нужные шрифты.

```
if (!QSqlDatabase::drivers().contains("SQLITE"))
    QSKIP("This test requires the SQLITE database driver");
```

- `QTEST(actual, testElement)` - `QTEST()` — это удобный макрос для `QCOMPARE()`, который сравнивает значение `actual` с элементом `testElement` из данных теста. Если такого элемента нет, тест завершается с ошибкой. В остальном `QTEST()` работает точно так же, как `QCOMPARE()`. Вместо того чтобы писать:

```
QFETCH(QString, myString);  
QCOMPARE(QString("hello").toUpper(), myString);
```

вы можете написать:

```
QTEST(QString("hello").toUpper(), "myString");
```

- `QTEST_APPLESS_MAIN(TestClass)` - Реализует функцию `main()`, которая выполняет все тесты в `TestClass`.
Работает так же, как `QTEST_MAIN()`, но не создаёт экземпляр объекта `QApplication`. Используйте этот макрос для очень простых автономных тестов без графического интерфейса.
- `QTEST_GUILESS_MAIN(TestClass)` - Реализует функцию `main()`, которая создает экземпляр `QCoreApplication` объекта и `TestClass` и выполняет все тесты в порядке их определения. Используйте этот макрос для создания автономных исполняемых файлов.
Работает как `QTEST_MAIN()`, но создаёт экземпляр `QCoreApplication` вместо `QApplication` объекта. Используйте этот макрос, если вашему тестовому примеру не нужна функциональность, предлагаемая `QApplication`, но цикл обработки событий всё равно необходим.
- `QTEST_MAIN(TestClass)` - Реализует функцию `main()`, которая создает экземпляр объекта приложения и `TestClass`, а также выполняет все тесты в том порядке, в котором они были определены. Используйте этот макрос для создания автономных исполняемых файлов.
- `QTRY_COMPARE_WITH_TIMEOUT(actual, expected, timeout)` - Макрос `QTRY_COMPARE_WITH_TIMEOUT()` аналогичен `QCOMPARE()`, но выполняет многократное сравнение значений `actual` и `expected` до тех пор, пока они не станут равны или не истечёт время `timeout` (в миллисекундах). Между каждым сравнением будут обрабатываться события. Если истечёт время ожидания, в журнале тестирования будет зафиксирован сбой, и тест не будет выполнен.

- `QTRY_VERIFY_WITH_TIMEOUT(condition, timeout)` - Макрос `QTRY_VERIFY_WITH_TIMEOUT()` аналогичен `QVERIFY()`, но проверяет `condition` несколько раз, пока условие не станет истинным или не истечёт время `timeout` (в миллисекундах). Между каждой проверкой будут обрабатываться события. Если истечёт время ожидания, в журнале тестирования будет зафиксирован сбой, и тест не будет выполнен.
- `QVERIFY(condition)` - Макрос `QVERIFY()` проверяет, является ли `condition` истинным или ложным. Если это так, выполнение продолжается. Если нет, в журнале тестирования фиксируется сбой, и тест дальше не выполняется.
Вы можете использовать `QVERIFY2()`, если это целесообразно и полезно для добавления дополнительной информации в отчёт о неудачном тестировании.
- `QVERIFY2(condition, message)` - Макрос `QVERIFY2()` работает точно так же, как `QVERIFY()`, за исключением того, что он выводит `message` при значении `condition` `false`. `message` — это обычная C-строка. Сообщение также может быть получено в результате вызова функции, которая возвращает обычную C-строку, например `qPrintable()`.
- `QVERIFY_EXCEPTION_THROWN(expression, exceptiontype)` - Макрос `QVERIFY_EXCEPTION_THROWN` выполняет `expression` и пытается перехватить исключение, вызванное в `expression`.

Возможны несколько вариантов:

Если `expression` генерирует исключение, которое совпадает с `exceptiontype` или является производным от `exceptiontype`, то выполнение продолжится.

В противном случае, если `expression` не генерирует исключение или генерируемое исключение является производным от `std::exception`, то в журнале тестирования будет зафиксирован сбой и макрос досрочно завершит работу (из внешней функции).

Если генерируемое исключение не является производным ни от `std::exception`, ни от `exceptiontype`, то в журнале тестирования будет зафиксирован сбой и исключение будет сгенерировано повторно. Это позволяет избежать проблем, например, с исключениями отмены `pthread`.

- `QWARN(message)` - Добавляет `message` в качестве предупреждения в журнал тестирования. Этот макрос можно использовать в любом месте ваших тестов.

С другими макросами вы можете познакомиться на странице документации здесь: <https://doc.qt.io/archives/qt-5.15/qtest.html>

4.4 Qt Quick Test

Qt Quick Test — это платформа для модульного тестирования приложений QML. Тестовые сценарии записываются в виде функций JavaScript в формате TestCase.

Листинг 8

```
1. import QtQuick 2.3
2. import QtTest 1.0
3.
4. TestCase {
5.     name: "MathTests"
6.
7.     function test_math() {
8.         compare(2 + 2, 4, "2 + 2 = 4")
9.     }
10.
11.    function test_fail() {
12.        compare(2 + 2, 5, "2 + 2 = 5")
13.    }
14. }
```

Функции, имена которых начинаются с `test_`, рассматриваются как тестовые сценарии, которые необходимо выполнить.

Из-за особенностей работы свойств JavaScript порядок, в котором вызываются тестовые функции, непредсказуем. Чтобы повысить предсказуемость, среда тестирования сортирует функции по возрастанию имени. Это может помочь, если нужно выполнить два теста по порядку.

4.4.1 Создание модульного теста QtQuick

- Запустите среду разработки Qt Creator;
- Файл → New Project;
- Выбрать шаблон Test Project → Qt Quick Test Project → Выбрать;
- Название (например) test_suite_1 → Далее;
- Имя теста test_case_1, Отметить: Generate setup code и Создать код инициализации и очистки, Система сборки CMake, Далее;
- Выбрать комплект, Далее, Завершить.

Будет создан проект, содержащий 4 файла:

Листинг 9 - Файл setup.h

```
1.  #ifndef SETUP_H
2.  #define SETUP_H
3.
4.  #include <QObject>
5.  #include <QQmlEngine>
6.
7.  class Setup : public QObject
8.  {
9.      Q_OBJECT
10. public:
11.     Setup() = default;
12.
13. public slots:
14.     void applicationAvailable();
15.     void qmlEngineAvailable(QQmlEngine *engine);
16.     void cleanupTestCase();
17. };
18.
19. #endif // SETUP_H
```

Листинг 10 - Файл main.cpp

```
1.  #include <QtQuickTest/quicktest.h>
```

```
2.  #include "setup.h"
3.
4.  QUICK_TEST_MAIN_WITH_SETUP(example, Setup)
```

Листинг 11 - Файл setup.cpp

```
1.  #include "setup.h"
2.
3.  void Setup::applicationAvailable()
4.  {
5.      // custom code that does not require QQmlEngine
6.  }
7.
8.  void Setup::qmlEngineAvailable(QQmlEngine *engine)
9.  {
10.     // custom code that needs QQmlEngine, register QML types, add
       import paths,...
11. }
12.
13. void Setup::cleanupTestCase()
14. {
15.     // custom code to clean up before destruction starts
16. }
```

Листинг 12 - Файл теста tst_test_case1.qml

```
1.  import QtQuick 2.15
2.  import QtTest 1.0
3.
4.  TestCase {
5.      name: "test_case1"
6.
7.      function initTestCase() {
8.          // code to be executed before the first test function
9.      }
```

```

10.
11.     function init() {
12.         // code to be executed before each test function
13.     }
14.
15.     function cleanupTestCase() {
16.         // code to be executed after the last test function
17.     }
18.
19.     function cleanup() {
20.         // code to be executed after each test function
21.     }
22.
23.     function test_case1() {
24.         compare(1 + 1, 2, "sanity check");
25.         verify(true);
26.     }
27. }

```

Вывод программы

```

QML debugging is enabled. Only use this in a safe environment.
***** Start testing of example *****
Config: Using QtTest library 6.10.0, Qt 6.10.0 (x86_64-
little_endian-lp64 shared (dynamic) release build; by GCC 11.4.1
20231218 (Red Hat 11.4.1-3)), fedora 43
PASS   : example::test_case1::initTestCase()
PASS   : example::test_case1::test_case1()
PASS   : example::test_case1::cleanupTestCase()
Totals: 3 passed, 0 failed, 0 skipped, 0 blacklisted, 3ms
***** Finished testing of example *****

```

4.4.2 Тесты, основанные на данных

Данные таблицы можно передать в тест, используя имя функции, которое заканчивается на "_data". Кроме того, функцию `init_data()` можно использовать для передачи тестовых данных по умолчанию для всех тестовых функций без соответствующей функции "_data" в типе `TestCase`:

Листинг 13

```
1.  import QtQuick 2.0
2.  import QtTest 1.2
3.
4.  TestCase {
5.      name: "DataTests"
6.
7.      function init_data() {
8.          return [
9.              {tag:"init_data_1", a:1, b:2, answer: 3},
10.             {tag:"init_data_2", a:2, b:4, answer: 6}
11.         ];
12.     }
13.
14.     function test_table_data() {
15.         return [
16.             {tag: "2 + 2 = 4", a: 2, b: 2, answer: 4 },
17.             {tag: "2 + 6 = 8", a: 2, b: 6, answer: 8 },
18.         ]
19.     }
20.
21.     function test_table(data) {
22.         //data comes from test_table_data
23.         compare(data.a + data.b, data.answer)
24.     }
25.
26.     function test_default_table(data) {
```

```

27.      //data comes from init_data
28.      compare(data.a + data.b, data.answer)
29.  }
30. }

```

Среда тестирования будет перебирать все строки в таблице и передавать каждую из них в тестовую функцию. Столбец tag является специальным — он выводится средой тестирования в случае сбоя в строке, чтобы тестировщик мог определить, какой из тестов завершился неудачно.

4.4.3 Контрольные показатели

Функции, имена которых начинаются с "benchmark_", будут запускаться несколько раз с помощью фреймворка Qt benchmark, а среднее время выполнения будет отображаться в результатах. Это эквивалентно использованию макроса QBENCHMARK в C++-версии QTestLib.

Листинг 14

```

1.  TestCase {
2.      id: top
3.      name: "CreateBenchmark"
4.
5.      function benchmark_create_component() {
6.          let component = Qt.createComponent("item.qml")
7.          let obj = component.createObject(top)
8.          obj.destroy()
9.          component.destroy()
10.     }
11. }

```

```

RESULT : CreateBenchmark::benchmark_create_component:
23 msec per iteration (total: 60, iterations: 256)
PASS   : CreateBenchmark::benchmark_create_component()

```

4.4.4 Имитация событий клавиатуры и мыши

Методы `keyPress()`, `keyRelease()` и `keyClick()` можно использовать для имитации событий клавиатуры в модульных тестах. События передаются элементу QML, находящемуся в данный момент в фокусе. Вы можете передать либо значение перечисления `Qt.Key`, либо символ `latin1` (строку длиной в один символ)

Листинг 15

```
1. Rectangle {
2.     width: 50; height: 50
3.     focus: true
4.
5.     TestCase {
6.         name: "KeyClick"
7.         when: windowShown
8.
9.         function test_key_click() {
10.             keyClick(Qt.Key_Left)
11.             keyClick("a")
12.             //...
13.         }
14.     }
15. }
```

Методы `mousePress()`, `mouseRelease()`, `mouseClick()`, `mouseDoubleClickSequence()` и `mouseMove()` можно использовать для имитации событий мыши аналогичным образом.

Если ваш тест создаёт другие окна, возможно, эти окна станут активными и перетянут фокус с окна `TestCase`. Чтобы окно `TestCase` оставалось активным, используйте следующий код:

```
testCase.Window.window.requestActivate()
tryCompare(testCase.Window.window, "active", true)
```

С документацией по свойствам QML типа TestCase можно познакомиться на этой странице: <https://doc.qt.io/qt-6/qml-qttest-testcase.html>

4.4.5 Другие типы QML для модульного тестирования

- **SignalSpy QML Type** - Позволяет анализировать излучение сигналов. Страница документации: <https://doc.qt.io/qt-6/qml-qttest-signalspy.html>
- **TouchEventSequence** - используется для создания и отправки сенсорных событий при тестировании. Страница документации: <https://doc.qt.io/qt-6/qml-qttest-toucheventsequence.html>

4.5 Постановка задачи

Выполнить модульное тестирование одного модуля C++ и одного QML-модуля своего курсового проекта.

4.6 Результаты

Предоставляются в виде двух проектов модульных тестов для C++ и QML модулей.

5. Документирование программного кода

5.1 Специальные блоки комментариев

Специальный блок комментариев - это блок комментариев в стиле C или C++ с некоторыми дополнительными пометками (тегами), которые воспринимаются Doxygen, как фрагмент структурированного текста, который должен быть включен в создаваемую документацию.

Для каждой сущности в коде существует два (или в некоторых случаях три) типа описаний, которые вместе образуют документацию для этой сущности: **краткое описание** (brief description) и **подробное описание** (detailed description). Оба не являются обязательными.

Для методов и функций существует также третий тип описания, так называемое **описание в теле** ("in body" description), которое состоит из объединения всех блоков комментариев, найденных в теле метода или функции.

Допускается наличие более одного краткого или подробного описания (но не рекомендуется, поскольку порядок, в котором описания будут отображаться, не указан).

Как следует из названия, краткое описание - это короткая однострочная информация, в то время как подробное описание содержит более длинную и подробную документацию. Есть несколько способов создания блоков подробного описания.

5.2 Оформление блока подробного описания

5.2.1 Первый вариант

Вариант оформления в стиле Qt (предпочтительный) — к комментариям в стиле C добавляется восклицательный знак. Обратите внимание, что среда программирования Qt Creator автоматизирует процесс документирования кода.

```
/*!  
 * ... текст ...  
 */
```

Промежуточные звездочки не обязательны:

```
/*!  
 ... текст ...  
 */
```

5.2.2 Второй вариант

Основан на использовании блока, состоящего, как минимум, из двух строк комментариев в стиле C++, где каждая строка начинается с дополнительной косой черты или восклицательного знака.

Например:

```
///  
/// ... текст ...  
///
```

Или

```
///  
///! ... текст ...  
///!
```

5.3 Оформление блока краткого описания

Для краткого описания также есть несколько возможностей. Можно использовать команду `\brief` с одним из приведенных выше блоков комментариев. Эта команда заканчивается в конце абзаца, поэтому подробное описание следует после пустой строки.

```
/*! \brief Краткое описание.  
* Продолжение краткого описания.  
*  
* Здесь начинается подробное описание.  
*/
```

Возможны также несколько вариантов без использования команды `\brief`, но в сочетании блоков `brief description` и `detailed description`.

5.3.1 Первый вариант

```
/// Краткое описание.  
/** Подробное описание. */
```

Или

```
//! Краткое описание.  
/** Подробное описание. */
```

5.3.2 Второй вариант

```
//! Краткое описание.  
  
//! Подробное описание  
//! начинается здесь.
```

Обратите внимание на пустую строку в последнем примере, которая требуется для отделения краткого описания от блока, содержащего подробное описание. (Для параметра `JAVADOC_AUTOBRIEF` в этом случае также должно быть установлено значение `NO`).

5.4 Однострочные комментарии для элементов

Если нужно задокументировать элементы файла, структуры, объединения, класса или перечисления, **необходимо поместить блок документации после элемента, а не перед ним**. Для этого вам необходимо поместить дополнительный маркер < в блок комментариев. Обратите внимание, что аналогичный подход применяется и для параметров функции. Например, в стиле Qt:

```
int var; /*!< Подробное описание после элемента */
```

Или

```
int var; /**< Подробное описание после элемента */
```

Но чаще всего требуется лишь указать краткое описание

```
int var; /*!< Краткое описание после элемента
```

Или

```
int var; ///  
Краткое описание после элемента
```

5.5 Пример документирования класса

```
1.  ///  
2.  /*!  
3.      Более подробное описание класса.  
4.  */  
5.  
6.  class QTeststyle_Test
```

```

7.  {
8.  public:
9.
10.  ///  
11.  /*!  
12.      enum TEnum {  
13.          TVal1, /*!  
14.          TVal2, /*!  
15.          TVal3  /*!  
16.      }  
17.      ///  
18.      /*!  
19.      *enumPtr,  
20.      ///  
21.      /*!  
22.      enumVar;  
23.  
24.      ///  
25.      /*!  
26.          Более подробное описание конструктора.  
27.      */  
28.      QTstyle_Test();  
29.  
30.      ///  
31.      /*!  
32.          Боле подробное описание деструктора.  
33.      */  
34.      ~QTstyle_Test();  
35.  
36.      ///  
37.      /*!  
38.          \param a целочисленный аргумент.

```

```

39.      \param s константный указатель на символьную строку.
40.      \return Результаты тестирования
41.      \sa QTeststyle_Test(), ~QTeststyle_Test(), testMeToo() and
      publicVar()
42.      */
43.      int testMe(int a,const char *s);
44.
45.      ///! Чисто виртуальный метод.
46.      /*!
47.      \sa testMe()
48.      \param c1 первый аргумент.
49.      \param c2 второй аргумент.
50.      */
51.      virtual void testMeToo(char c1,char c2) = 0;
52.
53.      ///! Общедоступная переменная.
54.      /*!
55.      Описание.
56.      */
57.      int publicVar;
58.
59.      ///! Указатель на функцию.
60.      /*!
61.      Описание.
62.      */
63.      int (*handler)(int a,int b);
64. };

```

Так будет выглядеть в документации

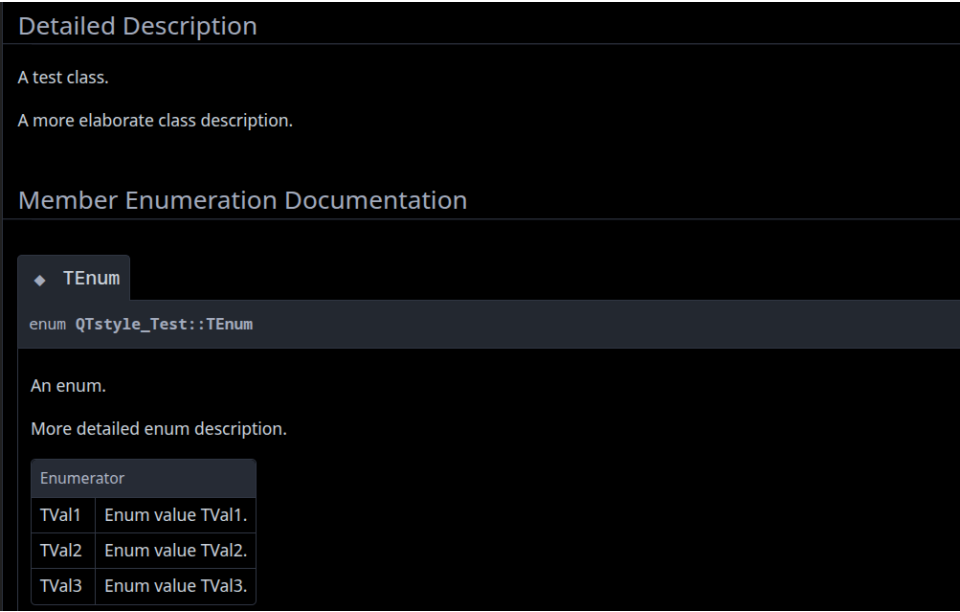


Рисунок 8

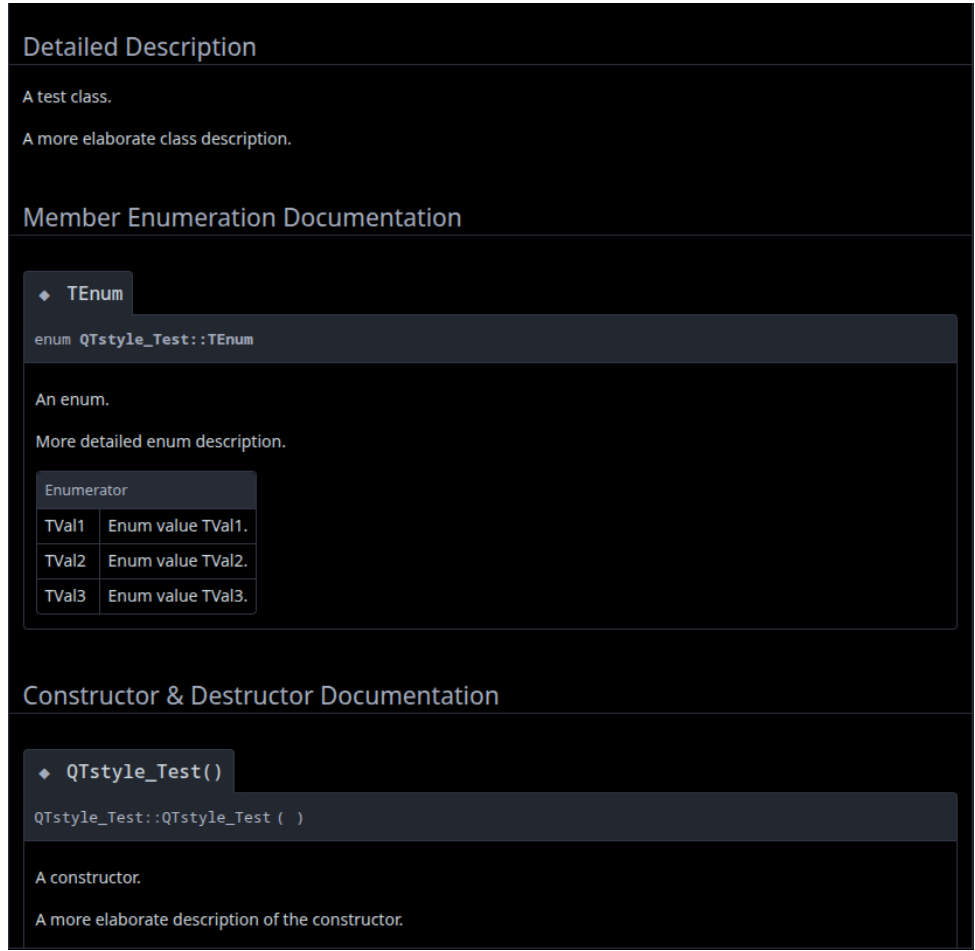


Рисунок 9

5.6 Детальное описание метода класса

Для подробного описания методов могут применяться следующие специальные команды:

- `\param` - Параметры передаваемые функции. Пример:

`\param name` Имя пользователя

Может применяться с опциональным атрибутом «направление». Этот атрибут может иметь три значения: `[in]`, `[out]`, `[in, out]`. Например:

```
/*! Копирует содержимое из исходной области памяти в целевую область  
память  
    \param[out] dest Целевая область памяти  
    \param[in] src Исходная область памяти  
    \param[in] n Количество байтов, которые необходимо скопировать  
*/  
void memcpy(void *dest, const void *src, size_t n);
```

- `\return` - Возвращаемое функцией значение. Пример:

`\return` Информация о пользователе

- `\throw` - Исключения выбрасываемые функцией. Пример:

`\throw DatabaseError` Если произошла ошибка при подключении к базе данных

- `\sa` - Начинается абзац, в котором может быть указана одна или несколько перекрестных ссылок на классы, функции, методы, переменные, файлы или URL.

5.7 Оформление шапки кода

Главный файл программы обычно содержит «шапку» с подробным описанием проекта (не обязательно, если эта информация документируется в отдельном пользовательском файле).

```
1.  /*!  
2.   * \brief Pretty nice class.  
3.   * \details This class is used to demonstrate a number of section  
   commands.  
4.   * \author John Doe  
5.   * \author Jan Doe  
6.   * \version 4.1a  
7.   * \date 1990-2011  
8.   * \pre First initialize the system.  
9.   * \bug Not all memory is freed when deleting an object of this  
   class.  
10.  * \warning Improper use can crash your application  
11.  * \copyright GNU Public License.  
12.  */  
13. class SomeNiceClass {};
```

Так будет выглядеть в документации

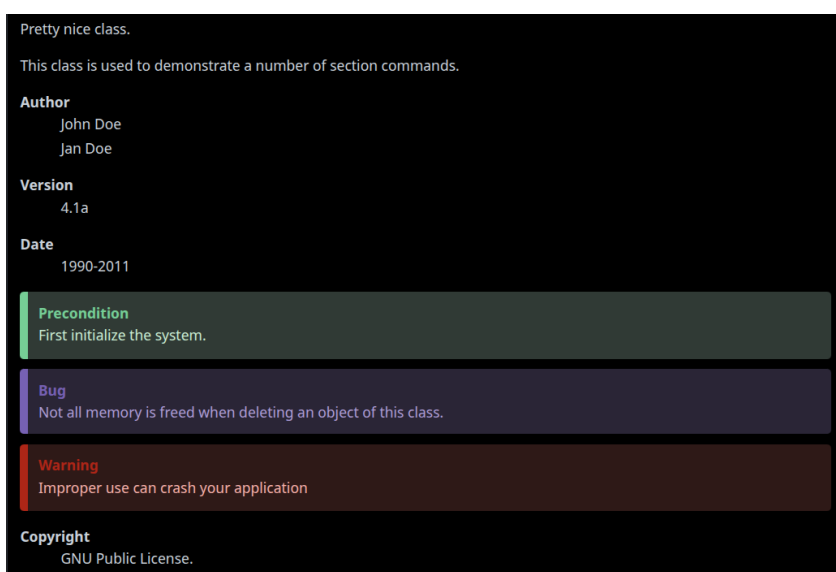


Рисунок 10

5.8 Структурные команды

Иногда удобнее размещать документацию в другом месте. Doxygen позволяет размещать блоки документации практически где угодно (за исключением тела функции или обычного блока комментариев в стиле C).

1. Блок документации, который не размещается непосредственно перед (или после) элемента, содержит структурные команды внутри блока документации. **Это может привести к дублированию информации.** Поэтому на практике следует избегать использования структурных команд, если только этого не требуют исключительные обстоятельства.
2. Структурные команды начинаются с обратной косой черты (\). Далее следует имя команды и один или несколько параметров.

К структурным командам относятся:

- \struct для документирования C-структуры.
- \union для документирования объединения.
- \enum для документирования типа перечисления.
- \fn для документирования функции.
- \var для документирования переменной, typedef или значения перечисления.
- \def для документирования #define.
- \typedef для документирования определения типа.
- \file для документирования файла.
- \namespace для документирования пространства имён.
- \package для документирования пакета Java.
- \interface для документирования интерфейса IDL.

Применять структурные команды — не рекомендуется!

Подробные сведения и примеры оформления структурных команд см. в документации: <https://www.doxygen.nl/manual/docblocks.html>

5.9 Страницы документации

- Документирование кода: <https://www.doxygen.nl/manual/docblocks.html>
- Настройка выходных данных:
<https://www.doxygen.nl/manual/customize.html>

- Дополнительная документация: <https://www.doxygen.nl/manual/additional.html>
- Выходные форматы: <https://www.doxygen.nl/manual/output.html>
- Автоматическая генерация ссылок: <https://www.doxygen.nl/manual/autolink.html>
- Поддержка Markdown: <https://www.doxygen.nl/manual/markdown.html>
- HTML — команды: <https://www.doxygen.nl/manual/htmlcmds.html>

5.10 Документирование классов QML

Файлы QML можно документировать в системе Doxygen с помощью фильтра Doxyqml. Он интегрируется как фильтр ввода Doxygen, превращая файлы .qml в псевдо-C++, который Doxygen может использовать для генерации документации.

5.10.1 Установка Doxyqml

```
sudo dnf install pip3
pip3 install doxyqml
```

5.10.2 Настройка Doxygen

Чтобы настроить Doxygen на использование Doxyqml, необходимо внести несколько изменений в файл конфигурации Doxygen.

Примечание. Если у вас уже есть файл конфигурации, скопируйте его, чтобы сохранить предыдущие настройки.

Добавьте расширение .qml к ключу FILTER_PATTERNS:

```
FILTER_PATTERNS = *.qml=doxyqml
```

Примечание: в Windows Doxyqml устанавливается в папку Scripts вашей установки Python. Если эта папка не находится в PATH, добавьте её или используйте полный путь к Doxyqml (но это менее удобно для разных компьютеров)

Добавьте расширение .qml к FILE_PATTERNS:

```
FILE_PATTERNS = *.qml
```

Начиная с Doxygen 1.8.8, вы также должны добавить расширение .qml в EXTENSION_MAPPING:

```
EXTENSION_MAPPING = qml=C++
```

Укажите путь к QML-файлам:

```
INPUT = path/to/qml/files
```

5.10.3 Документирование типов. Функции

QML частично типизирован: функции нетипизированы, а свойства и сигналы типизированы. Doxyqml позволяет определять типы, если они отсутствуют или недостаточно точны.

Типы в документации можно определить следующим образом:

```
1.  /*!  
2.   * Create a user  
3.   * \param type:string firstname User firstname  
4.   * \param type:string lastname User lastname  
5.   * \param type:int User age  
6.   * \return type:User The User object  
7.  */  
8.  function createUser(firstname, lastname, age);
```

5.10.4 Свойства

Свойства QML типизированы, поэтому Doxyqml использует их по умолчанию. Тем не менее вы можете переопределить тип, используя тот же синтаксис type:<name>. Это полезно для документирования псевдонимов свойств:

```
1.  /*! type:string The user lastname */  
2.  property alias lastname: someObject.text
```

5.10.5 Сигналы

Сигналы QML типизированы, поэтому нет необходимости использовать синтаксис `type:<name>` для документирования их параметров. Использование синтаксиса `type:<name>` в документации сигналов не сработает: Doxygen не удалит его, а Doxygen перепутает его с именем параметра.

```
1.  /**
2.   * User just logged in
3.   * @param user The user which logged in
4.   */
5.  signal loggedIn(User user)
```

5.10.6 Извлечение внутренних элементов

Элементы QML с идентификатором экспортируются как приватные переменные-члены. Если вы установите для ключей `EXTRACT_ALL` и `EXTRACT_PRIVATE` Doxygen значение `YES`, то эти элементы будут отображаться в сгенерированной документации.

5.10.7 Пример документации QML-файла

```
1.  /*!
2.   * \file ButtonWidget.qml
3.   * \brief Виджет кнопки с расширенными возможностями стилизации.
4.   *
5.   * Этот компонент реализует настраиваемую кнопку с поддержкой:
6.   * - разных состояний (нормальное, наведение, нажатие);
7.   * - анимации переходов;
8.   * - кастомного текста и иконок.
9.   *
10.  * \author Иван Петров <ivan.petrov@example.com>
11.  * \version 1.2
12.  * \date 2025-11-15
```

```

13.  * \copyright Copyright 2025, Example Corp. Все права защищены.
14.  */
15.
16.  import QtQuick 2.15
17.  import QtQuick.Controls 2.15
18.
19.  /*!
20.  * \class ButtonWidget
21.  * \brief Основной компонент кнопки.
22.  *
23.  * Кнопка реагирует на нажатия и наведение курсора, поддерживает:
24.  * – настройку цвета фона и текста;
25.  * – отображение иконки слева от текста;
26.  * – анимацию при наведении.
27.  *
28.  * \snippet ButtonWidget.qml UsageExample
29.  */
30.  Rectangle {
31.      id: root
32.      width: 120
33.      height: 40
34.      radius: 6
35.      color: "#4CAF50"
36.
37.      /*!
38.      * \property string text
39.      * \brief Текст, отображаемый на кнопке.
40.      *
41.      * По умолчанию – пустая строка.
42.      * \par Пример:
43.      * \code
44.      * text: "Отправить"
45.      * \endcode

```

```

46.      */
47.      property string text: ""
48.
49.      /*!
50.      * \property url iconSource
51.      * \brief Путь к изображению иконки.
52.      *
53.      * Если задано, иконка отображается слева от текста.
54.      * \par Пример:
55.      * \code
56.      * iconSource: "images/send.png"
57.      * \endcode
58.      */
59.      property url iconSource: ""
60.
61.      /*!
62.      * \property bool enabled
63.      * \brief Флаг активности кнопки.
64.      *
65.      * Если `false`, кнопка становится серой и не реагирует на
        клики.
66.      * \default true
67.      */
68.      property bool enabled: true
69.
70.      /*!
71.      * \signal clicked()
72.      * \brief Сигнал, испускаемый при нажатии на кнопку.
73.      *
74.      * Подключите обработчик, чтобы выполнить действие при клике.
75.      * \par Пример подключения:
76.      * \code
77.      * onClicked: console.log("Кнопка нажата!")

```

```

78.      * \endcode
79.      */
80.      signal clicked()
81.
82.      /*!
83.      * \method reset()
84.      * \brief Сбрасывает текст кнопки к пустому значению.
85.      *
86.      * Вызывает обновление визуального состояния.
87.      * \par Пример вызова:
88.      * \code
89.      * button.reset()
90.      * \endcode
91.      */
92.      function reset() {
93.          root.text = ""
94.      }
95.
96.      // Внутренняя логика (не документируется)
97.      MouseArea {
98.          anchors.fill: parent
99.          enabled: root.enabled
100.         onClicked: root.clicked()
101.     }
102.
103.     Text {
104.         id: label
105.         anchors.centerIn: parent
106.         text: root.text
107.         color: "white"
108.         font.pixelSize: 14
109.     }
110.

```

```

111.     Image {
112.         id: icon
113.         source: root.iconSource
114.         anchors.verticalCenter: parent.verticalCenter
115.         anchors.left: parent.left
116.         leftMargin: 10
117.         visible: root.iconSource != ""
118.     }
119. }
120.
121. /*!
122.  * \page UsageExample
123.  * \section Пример использования ButtonWidget
124.  *
125.  * Вставьте компонент в ваш QML-файл:
126.  *
127.  * \code
128.  * ButtonWidget {
129.  *     id: sendButton
130.  *     text: "Отправить"
131.  *     iconSource: "images/send.png"
132.  *     onClicked: console.log("Отправлено!")
133.  * }
134.  * \endcode
135.  */

```

5.10.8 Пояснения к тегам Doxyqml

- \file — имя файла и краткое описание.
- \class — описание компонента QML как класса.
- \property — документация свойства (с типом и примером).
- \signal — описание сигнала (с примером подключения).
- \method — документация функции/метода (с примером вызова).
- \snippet — ссылка на пример использования (см. раздел \page).

- \page — отдельная страница документации с примерами.
- \par — абзац внутри описания.
- \code /\endcode — блок кода (сохраняет форматирование).

Так будет выглядеть в документации

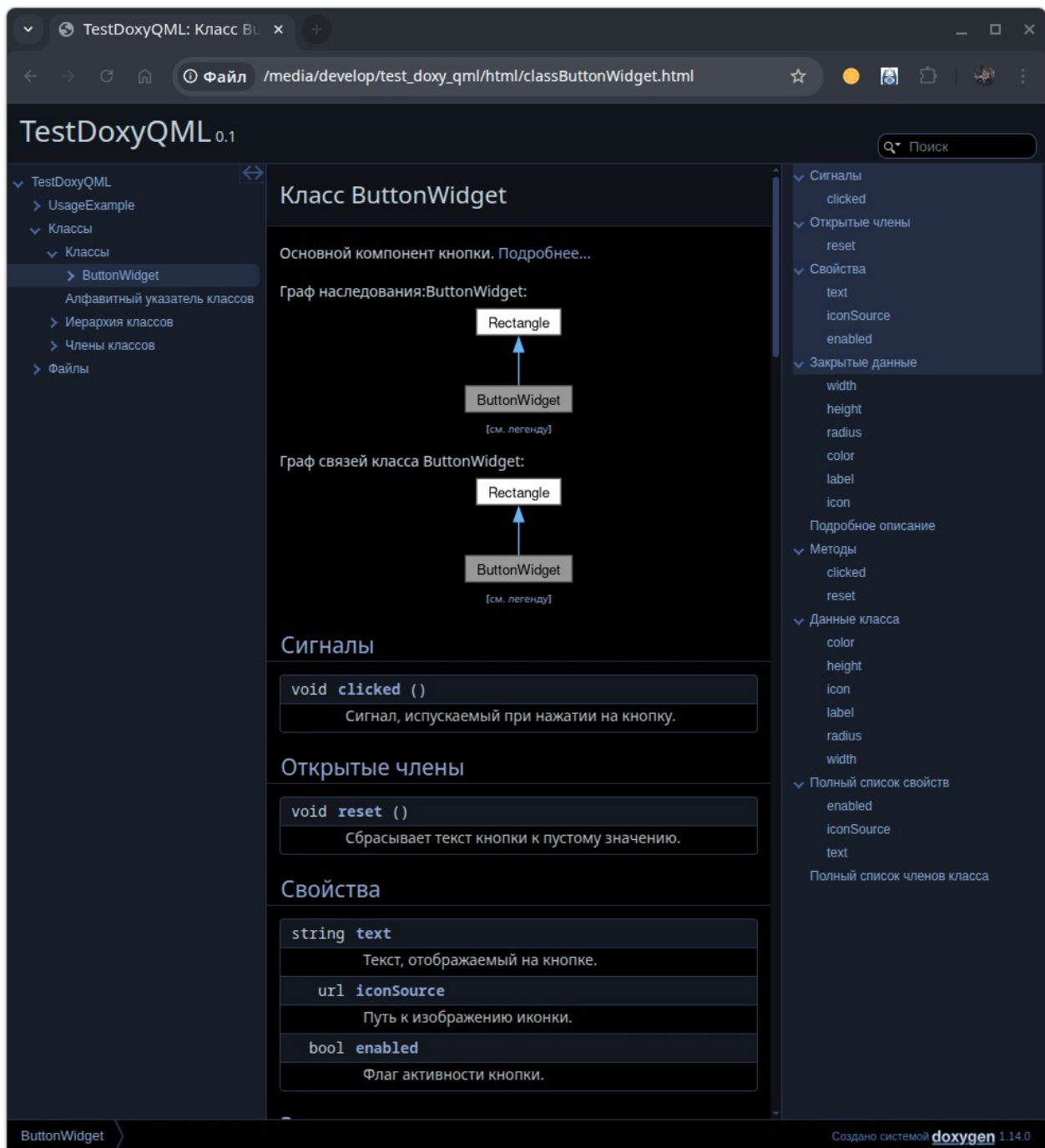


Рисунок 11

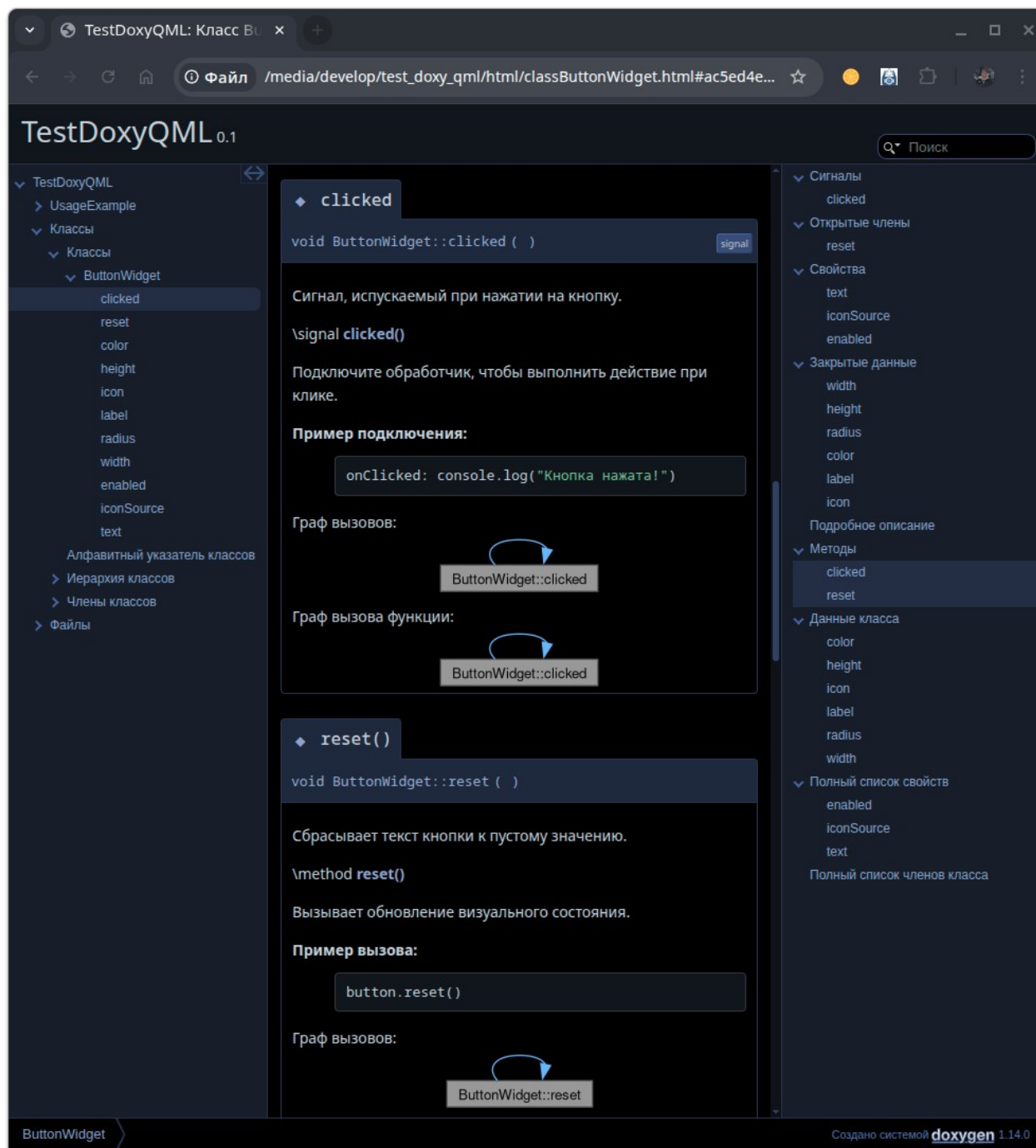


Рисунок 12

5.11 Постановка задачи

Создайте документацию Doxygen в стиле Qt для одного модуля QML и одного модуля C++. Сгенерируйте файлы документации в формате HTML.

5.12 Результаты

Предоставить преподавателю документацию модулей проекта в формате HTML.

Рекомендуемая литература

1. Старолетов С. М. Основы тестирования программного обеспечения : учебное пособие для СПО / С. М. Старолетов. — 3-е изд., стер. — Санкт-Петербург : Лань, 2023
2. Аниче М. Эффективное тестирование программного обеспечения / пер. с англ. А. Н. Киселева. – М.: ДМК Пресс, 2022
3. Куликов, С. С. Тестирование программного обеспечения. Базовый курс / С. С. Куликов. — 3-е изд. — Минск: Четыре четверти, 2020
4. Хориков В. Принципы юнит-тестирования. — СПб.: Питер, 2021